IF3270 PEMBELAJARAN MESIN

Tugas Besar 1

Feedforward Neural Network

Anggota 1	Muhammad Raihaan Perdana / 13523124
Anggota 2	Danendra Shafi Athallah / 13523136
Anggota 3	M. Abizzar Gamadrian / 13523155

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2025/2026

Daftar Isi

1. Pendahuluan 4
    1.1 Latar Belakang 4
    1.2 Tujuan 4
    1.3 Dataset 5
    1.4 Data Preprocessing 5
        1.4.1 Pembagian Data 5
        1.4.2 Encoding Fitur Kategorikal 5
        1.4.3 Normalisasi atau Standarisasi Fitur Numerik 6
        1.4.4 Penyesuaian Label Klasifikasi 6
2. Landasan Teori 6
    2.1 Arsitektur dan Struktur Kode 6
    2.2 Kelas FFNN 8
        2.2.1 Atribut Utama 8
        2.2.2 Method 8
        2.2.3 Method Utama 9
    2.3 Forward Propagation 9
        2.3.1 Penerapan Normalisasi 10
        2.3.2 Penerapan Fungsi Aktivasi 10
        2.3.3 Penerapan Dropout 10
        2.3.4 Proses Lapisan demi Lapisan 10
        2.3.5 Output Akhir 10
    2.4 Backward Propagation 10
        2.4.1 Inisialisasi Gradien 10
        2.4.2 Backward melalui setiap lapisan 11
        2.4.3 Gradient Loss Function 11
        2.4.4 Chain Rule per Layer 12
    2.5 Fungsi Aktivasi 12
    2.6 Loss Function 13
        2.6.1 Cross Entropy Loss 13
        2.6.2 Alasan Penggunaan untuk Klasifikasi 13
    2.7 Inisialisasi Bobot 13
        2.7.1 Pentingnya Weight Initialization 14
        2.7.2 Metode Inisialisasi yang Diuji 14
            2.7.2.1 Zero Initialization 14
            2.7.2.2 Uniform Initialization 14
            2.7.2.3 Normal Initialization 14
            2.7.2.4 Xavier Initialization 14
            2.7.2.5 He Initialization 14
        2.7.3 Pengaruh terhadap Proses Training 14
    2.8 Regularisasi 15
        2.8.1 Konsep Dasar Regularisasi 15
        2.8.2 L1 Regularization (Lasso) 15
        2.8.3 L2 Regularization (Ridge) 15

2.8.4 Parameter Lambda 15
2.9 Weight Update (Gradient Descent) 16
2.9.1 Gradient Descent Optimizer 16
2.9.2 Peran Learning Rate 16
2.10 Automatic Differentiation 16
2.10.1 Kelas Value 16
2.10.2 Overloading Operator 17
2.10.3 Fungsi Aktivasi 17
2.10.4 Computational Graph dan Topological Sort 17
2.10.5 Integrasi dengan FFNN 17
2.11 Adam Optimizer 17
2.12 RMSNorm 18
2.12.1 Implementasi Forward Pass 18
2.12.2 Parameter Gain 18
2.12.3 Backward Pass 18
2.12.4 Keunggulan RMSNorm 18
3. Pengujian 20
3.1 Setup Eksperimen 20
3.2 Evaluation Metrics 20
3.3 Pengaruh Depth dan Width 20
3.3.1 Variasi Width (Depth Tetap) 20
3.3.2 Variasi Depth (Width Tetap) 22
3.4 Pengaruh Fungsi Aktivasi 24
3.5 Pengaruh Learning Rate 28
3.6 Pengaruh Regularisasi 32
3.7 Pengaruh Inisialisasi Bobot 37
3.8 Pengaruh Normalisasi RMSNorm 41
3.9 Perbandingan dengan Sklearn MLP 44
4. Kesimpulan dan Saran 46
4.1 Kesimpulan 46
4.2 Saran 46
4.3 Daftar Pustaka 50
4.4 Lampiran 51

1. Pendahuluan

1.1 Latar Belakang

Machine learning saat ini telah menjadi salah satu teknologi yang sangat penting dalam berbagai bidang, terutama dalam prediksi dan klasifikasi data. Kemampuan machine learning untuk mempelajari pola dari data secara otomatis membuatnya sangat berguna dalam pengambilan keputusan berbasis data. Berbeda dengan metode statistik tradisional yang mengasumsikan hubungan linear antar variabel, machine learning mampu menangkap hubungan yang lebih kompleks dan non-linear dalam data.

Salah satu arsitektur machine learning yang sangat populer dan fundamental adalah neural network atau jaringan saraf tiruan. Neural network terinspirasi dari cara kerja otak manusia dalam memproses informasi. Keunggulan utama neural network terletak pada kemampuannya untuk mempelajari hubungan non-linear yang kompleks antara input dan output melalui lapisan-lapisan tersembunyi (hidden layers). Hal ini menjadikan neural network sangat efektif untuk berbagai tugas seperti klasifikasi, regresi, dan pengenalan pola.

Pada tugas besar ini, dipilih arsitektur Feedforward Neural Network (FFNN) sebagai objek implementasi. FFNN merupakan jenis neural network paling sederhana dimana informasi mengalir secara searah dari input menuju output tanpa adanya loop atau siklus. Meskipun relatif sederhana, FFNN menjadi fondasi penting untuk memahami arsitektur neural network yang lebih kompleks seperti convolutional neural network (CNN) dan recurrent neural network (RNN).

Selain itu, implementasi FFNN from scratch menggunakan Python dan NumPy tanpa menggunakan library machine learning seperti PyTorch atau TensorFlow memiliki tujuan edukasional yang sangat penting. Dengan mengimplementasikan sendiri mekanisme forward propagation dan backpropagation, mahasiswa dapat memahami secara mendalam bagaimana neural network bekerja secara internal. Pemahaman ini sulit diperoleh jika hanya menggunakan library yang sudah menyediakan implementasi siap pakai. Pengalaman mengimplementasikan gradient descent, weight initialization, dan berbagai fungsi aktivasi secara manual akan memberikan wawasan yang lebih baik tentang hyperparameter tuning dan troubleshooting model neural network.

1.2 Tujuan

Tujuan utama dari tugas besar ini adalah mengimplementasikan dan menganalisis Feedforward Neural Network (FFNN) untuk masalah klasifikasi biner pada dataset job placement atau employability. Secara spesifik, tujuan yang ingin dicapai meliputi:

- Mengimplementasikan FFNN from scratch dengan dukungan berbagai fungsi aktivasi, loss function, dan metode inisialisasi bobot.
- Memahami dan mengimplementasikan mekanisme backpropagation menggunakan chain rule.
- Melakukan eksperimen analisis pengaruh berbagai hyperparameter terhadap performa model.
- Membandingkan hasil implementasi dengan library sklearn MLP.
- (Bonus) Mengimplementasikan automatic differentiation, optimizer Adam, dan normalisasi RMSNorm.

1.3 Dataset

Dataset yang digunakan pada tugas ini adalah Global Student Placement & Salary Dataset (datasetml_2026.csv) yang berisi 10.000 data mahasiswa dengan 12 fitur. Target prediksi adalah kolom placement_status yang merupakan label kategoris status penempatan kerja mahasiswa.

Kolom	Deskripsi	Tipe
cgpa	Cumulative Grade Point Average (4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical
country	Negara kelulusan	Categorical
university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Float
communication_score	Skor kemampuan komunikasi	Float
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float
placement_status	Status penempatan kerja (TARGET)	Categorical

1.4 Data Preprocessing

Sebelum data dapat digunakan untuk melatih neural network, diperlukan tahapan data preprocessing untuk memastikan data dalam kondisi yang sesuai dan optimal untuk proses pelatihan. Tahapan-tahapan preprocessing yang umumnya dilakukan meliputi:

1.4.1 Pembagian Data

Data yang tersedia dibagi menjadi beberapa himpunan bagian, terutama training set dan validation set. Training set digunakan untuk melatih model, sementara validation set digunakan untuk mengevaluasi performa model selama proses pelatihan dan melakukan hyperparameter tuning. Pada beberapa kasus, test set juga dipisahkan untuk evaluasi akhir model. Pembagian ini biasanya dilakukan dengan perbandingan tertentu, misalnya 80:20 atau 70:15:15 untuk training, validation, dan test masing-masing.

1.4.2 Encoding Fitur Kategorikal

Dataset seringkali mengandung fitur-fitur kategorikal (categorical features) seperti jenjang pendidikan atau jenis pekerjaan. Neural network hanya dapat memproses data numerik, sehingga fitur kategorikal perlu dikonversi menjadi representasi numerik. Teknik yang umum digunakan meliputi Label Encoding (mengubah kategori menjadi angka berurutan) atau One-Hot Encoding (mengubah setiap kategori menjadi vektor biner).

1.4.3 Normalisasi atau Standarisasi Fitur Numerik

Fitur numerik dengan skala yang berbeda-beda dapat menyebabkan masalah dalam proses pelatihan neural network. Fitur dengan nilai yang sangat besar dapat mendominasi fitur dengan nilai kecil, sehingga mempengaruhi pembelajaran bobot. Normalisasi (mengubah data ke rentang tertentu, misalnya 0-1) atau standarisasi (mengubah data memiliki mean 0 dan standar deviasi 1) dilakukan untuk memastikan semua fitur berada dalam skala yang serupa, sehingga proses pelatihan menjadi lebih stabil dan cepat.

1.4.4 Penyesuaian Label Klasifikasi

Untuk masalah klasifikasi biner, label kelas (misalnya "employed" dan "not employed") perlu dikonversi menjadi format numerik yang sesuai, biasanya 0 dan 1. Hal ini diperlukan untuk perhitungan loss function dan interpretasi output model.

2. Landasan Teori

2.1 Arsitektur dan Struktur Kode

Implementasi FFNN pada tugas ini disusun dalam beberapa modul terpisah untuk menjaga keteraturan kode dan kemudahan pemeliharaan. Berikut adalah deskripsi struktur modul yang diimplementasikan:

```
src/
├── models/            # Modul model utama
│   ├── ffnn.py        # Kelas FFNN utama
│   ├── base.py        # BaseModel abstract class
│   └── autodiff_ffnn.py # Implementasi FFNN dengan autodiff
├── layers/            # Definisi lapisan neural network
│   ├── dense.py       # Dense (Fully Connected) layer
│   ├── input.py       # Input layer
│   ├── normalization.py # RMSNorm dan LayerNormalization
│   └── dropout.py     # Dropout layer
├── activations/       # Fungsi aktivasi
│   ├── relu.py        # ReLU
│   ├── sigmoid.py     # Sigmoid
│   ├── softmax.py     # Softmax
│   ├── tanh.py        # Tanh
│   ├── linear.py      # Linear (identity)
│   └── bonus_activations.py # LeakyReLU, ELU
├── losses/            # Fungsi loss
│   ├── binary_cross_entropy.py # BCE Loss
│   ├── categorical_cross_entropy.py # CCE Loss
│   └── mse.py         # MSE Loss
├── initializers/      # Inisialisasi bobot
│   ├── zero.py        # Zero initialization
```

```
│   ├── uniform.py     # Uniform initialization
│   ├── normal.py      # Normal (Gaussian) initialization
│   └── bonus_initializers.py # Xavier dan He initialization
├── regularizers/      # Regularisasi
│   ├── l1.py          # L1 (Lasso) regularization
│   └── l2.py          # L2 (Ridge) regularization
├── optimizers/        # Algoritma optimasi
│   ├── gradient_descent.py # Standard Gradient Descent
│   └── adam.py        # Adam optimizer
├── normalization/     # Teknik normalisasi
│   └── rmsnorm.py     # RMS Normalization
├── autodiff/          # Automatic differentiation
│   ├── engine.py      # Autodiff engine
│   └── value.py       # Value class untuk autodiff
└── utils/             # Utilitas tambahan
    ├── preprocessing.py # Data preprocessing dan EDA
    ├── metrics.py     # Metrik evaluasi
    ├── plotting.py    # Visualisasi
    ├── pipeline.py    # Pipeline training
    └── tune_hyperparams.py # Hyperparameter tuning
```

File / Modul	Deskripsi
ffnn.py	Kelas utama FFNN: core model dengan forward, backward, fit, predict
autodiff_ffnn.py	FFNN dengan automatic differentiation (Bonus)
dense.py / input.py	Representasi layer Dense dan Input
linear/relu/sigmoid/tanh/softmax.py	Fungsi aktivasi beserta turunannya
bonus_activations.py	Fungsi aktivasi tambahan (Bonus)
mse / binary_ce / categorical_ce.py	Implementasi loss function
zero / uniform / normal.py	Metode inisialisasi bobot
bonus_initializers.py	Xavier & He initialization (Bonus)
l1.py / l2.py	Regularisasi L1 dan L2
gradient_descent.py	Optimizer Gradient Descent
adam.py	Optimizer Adam (Bonus)
rmsnorm.py / normalization.py	Normalisasi RMSNorm (Bonus)

File / Modul	Deskripsi
engine.py / value.py	Engine autodiff: Value class dengan computational graph (Bonus)
preprocessing.py	Preprocessing dataset (encoding, scaling, splitting)
pipeline.py / train.py	Pipeline pelatihan dan evaluasi
plotting.py	Visualisasi distribusi bobot, gradien, dan loss curve
metrics.py	Metrik evaluasi model
io.py	Metode save dan load model
main.py	Entry point program
testing.ipynb	Notebook pengujian dan eksperimen

2.2 Kelas FFNN

2.2.1 Atribut Utama

Kelas FFNN memiliki atribut-atribut penting sebagai berikut:

- layer_sizes: List integer yang menentukan jumlah neuron pada setiap lapisan, termasuk input dan output layer.
- weights dan biases: List numpy array yang menyimpan parameter-model yang akan dipelajari selama proses pelatihan. Setiap elemen sesuai dengan satu lapisan.
- weight_gradients dan bias_gradients: List yang menyimpan gradien bobot dan bias untuk setiap lapisan.
- activation_functions: List string yang menentukan fungsi aktivasi untuk setiap lapisan (hidden dan output).
- learning_rate: Hyperparameter yang menentukan ukuran langkah dalam proses pembaruan bobot menggunakan gradient descent.
- loss_function: Nama fungsi loss yang digunakan ('mse', 'binary_cross_entropy', atau 'categorical_cross_entropy').
- regularizer: Dictionary yang menentukan tipe regularisasi ('l1' atau 'l2') dan nilai lambda.
- normalization: List yang menentukan jenis normalisasi untuk setiap lapisan ('rmsnorm', 'layernorm', atau None).
- dropout_rate:.float yang menentukan tingkat dropout untuk regularisasi.
- normalization_layers dan dropout_layers: List yang menyimpan objek layer untuk normalisasi dan dropout.
- layer_inputs dan layer_outputs: List yang menyimpan nilai intermediate untuk proses backward.
- is_fitted: Boolean flag yang menandakan apakah model sudah dilatih.

2.2.2 Method

Pada konstruktor, pengguna menentukan:

- Arsitektur jaringan melalui layer_sizes
- Fungsi aktivasi untuk setiap lapisan melalui activations
- Loss function, initializer, learning rate
- Konfigurasi regularisasi dan normalisasi
- Dropout rate

2.2.3 Method Utama

Kelas FFNN memiliki beberapa method utama:

- _initialize_parameters(): Menginisialisasi bobot, bias, normalization layers, dan dropout layers sesuai dengan konfigurasi yang diberikan.
- forward(X): Melakukan forward propagation dengan menghitung output dari setiap lapisan secara berurutan. Untuk setiap lapisan: (1) hitung z, (2) apply normalization jika ada, (3) apply fungsi aktivasi, (4) apply dropout jika bukan layer terakhir. Method ini menyimpan intermediate values untuk backward pass.
- backward(grad): Melakukan backpropagation untuk menghitung gradien loss terhadap setiap parameter. Dimulai dari output layer, menggunakan chain rule untuk propagasi gradien ke layer sebelumnya. Includes backward pass melalui normalization dan dropout layers, serta penambahan gradien regularisasi jika dikonfigurasi.
- train(X_train, y_train, X_val, y_val, batch_size, epochs, optimizer, patience): Method utama untuk melatih model. Menggunakan mini-batch gradient descent dengan shuffle pada setiap epoch. Menyediakan early stopping berdasarkan validation loss. Returns training history berisi train_loss dan val_loss per epoch.
- predict(X): Melakukan prediksi pada data baru. Untuk BCE loss, mengembalikan class labels (threshold 0.5). Untuk CCE loss, mengembalikan argmax dari probabilitas. Untuk MSE, mengembalikan raw predictions.
- save(filepath) dan load(filepath): Method untuk menyimpan dan memuat model menggunakan pickle, termasuk parameter normalisasi.
- plot_weight_distribution() dan plot_gradient_distribution(): Method untuk visualisasi distribusi bobot dan gradien.

2.3 Forward Propagation

Forward Propagation adalah proses menghitung output dari neural network dengan mengalirkan data secara maju dari lapisan input menuju lapisan output. Proses ini merupakan tahap pertama dalam setiap iterasi pelatihan dan juga digunakan saat melakukan prediksi dengan model yang sudah dilatih. Implementasi forward propagation pada kelas FFNN dapat dilihat di src/models/ffnn.py

Untuk setiap layer l, dilakukan operasi berikut:

Perhitungan Forward Pass

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

2.3.1 Penerapan Normalisasi

Jika konfigurasi menggunakan normalisasi (RMSNorm atau LayerNormalization), normalisasi diterapkan pada hasil perhitungan linear sebelum aktivasi. Hal ini menstabilkan pelatihan dengan menjaga distribusi aktivasi tetap konstan.

2.3.2 Penerapan Fungsi Aktivasi

Setelah normalisasi (jika ada), hasil z diteruskan ke fungsi aktivasi. Fungsi aktivasi memperkenalkan non-linearity ke dalam jaringan, memungkinkan neural network untuk mempelajari pola-pola yang kompleks dan non-linear.

2.3.3 Penerapan Dropout

Untuk lapisan tersembunyi (bukan lapisan output), dropout dapat diterapkan setelah fungsi aktivasi. Dropout secara acak meng-set beberapa neuron menjadi nol selama training untuk mencegah overfitting.

2.3.4 Proses Lapisan demi Lapisan

Proses forward propagation pada implementasi ini berlangsung secara berurutan untuk setiap lapisan:

1. Linear transformation: z = X * W + b (disimpan ke layer outputs)
2. Normalization: Jika dikonfigurasi, apply RMSNorm atau LayerNormalization
3. Activation function: Apply fungsi aktivasi sesuai konfigurasi (disimpan ke activations_outputs)
4. Dropout: Jika bukan layer terakhir dan dropout_rate > 0, apply dropout (disimpan ke layer_inputs sebagai input untuk layer berikutnya)

Semua nilai intermediate disimpan dalam list (layer_inputs, layer_outputs, activations_outputs) untuk digunakan dalam backward pass.

2.3.5 Output Akhir

Hasil akhir dari forward propagation adalah prediksi jaringan untuk input yang diberikan. Output disimpan dalam variabel current_output yang kemudian dikembalikan oleh method forward().

2.4 Backward Propagation

Backward Propagation (sering disingkat menjadi backprop) adalah algoritma fundamental yang memungkinkan neural network untuk belajar dari kesalahan prediksinya. Backprop menghitung gradien fungsi loss terhadap setiap bobot dan bias dalam jaringan. Implementasi backward propagation pada kelas FFNN dapat dilihat di src/models/ffnn.py

2.4.1 Inisialisasi Gradien

Proses backprop dimulai dengan menerima gradien dari loss function yang dihitung berdasarkan prediksi dan label sebenarnya. Gradien ini kemudian di-propagate mundur melalui jaringan.

2.4.2 Backward melalui setiap lapisan

Backward propagation berjalan dari lapisan terakhir (output) menuju lapisan pertama (input). Untuk setiap lapisan, dilakukan urutan operasi berikut:

1. Backward melalui activation function: Hitung gradien terhadap input pre-activation dengan mengalikan gradien output dengan turunan fungsi aktivasi. Untuk softmax dengan cross-entropy loss, gradien disederhanakan menjadi
2. Backward melalui normalization layer: Propagasi gradien melalui RMSNorm atau LayerNormalization. Ini melibatkan perhitungan gradien terhadap input yang sudah dinormalisasi.
3. Backward melalui dropout layer: Propagasi gradien dengan mengabaikan neuron yang di dropout saat training.
4. Perhitungan gradien terhadap bobot dan bias:

   Untuk bobot menggunakan np.dot(layer_input.T, activation_grad)

   Untuk bias menggunakan np.sum(activation_grad, axis=0)

5. Propagasi gradien ke layer sebelumnya: menggunakan np.dot(activation_grad, self.weights[i].T)

2.4.3 Gradient Loss Function

Turunan pertama loss terhadap output dihitung berdasarkan fungsi loss yang digunakan:

MSE

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Binary Cross-Entropy

$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$

y_i = Actual binary label (0 or 1)

$\hat{y}_i$ = Predicted value of y_i

n = Batch size

Categorical Cross-Entropy

$$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij}\log\hat{y}_{ij})$$

y_{ij} = Actual value of instance i for class j

$\hat{y}_{ij}$ = Predicted value of y_{ij}

C = Number of classes

n = Batch size

2.4.4 Chain Rule per Layer

Untuk setiap layer l (dari output ke input), gradien dihitung sebagai berikut:

Backpropagation per Layer

$$\frac{\partial Loss}{\partial w_{j_1 k_1}} = \frac{\partial Loss}{\partial k_{1out}} \times \frac{\partial k_{1out}}{\partial k_{1in}} \times \frac{\partial k_{1in}}{\partial w_{j_1 k_1}}$$

2.5 Fungsi Aktivasi

Berikut adalah fungsi aktivasi yang diimplementasikan beserta turunannya:

Aktivasi	Definisi	Turunan
Linear	$Linear(x) = x$	$\frac{d(Linear(x))}{dx} = 1$
ReLU	$ReLU(x) = \max(0, x)$	$\frac{d(ReLU(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1-\sigma(x))$

Aktivasi	Definisi	Turunan
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}}\right)^2$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(softmax(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\vec{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(softmax(\vec{x})_i)}{\partial x_j} = softmax(\vec{x})_i(\delta_{i,j} - softmax(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$

2.6 Loss Function

Loss function (atau cost function) mengukur seberapa baik performansi neural network dalam melakukan prediksi. Loss function mengembalikan nilai skalar yang merepresentasikan "harga" dari prediksi yang salah semakin kecil nilainya, semakin baik prediksi model. Tujuan utama pelatihan neural network adalah meminimalkan nilai loss function ini.

2.6.1 Cross Entropy Loss

Untuk masalah klasifikasi, Cross Entropy Loss merupakan pilihan yang paling umum dan sangat disarankan. Cross entropy mengukur perbedaan antara dua distribusi probabilitas: distribusi prediksi model dan distribusi kebenaran ground-truth.

2.6.2 Alasan Penggunaan untuk Klasifikasi

Cross entropy loss sangat cocok untuk masalah klasifikasi karena beberapa alasan:

1. Sifat probabilistik: Cross entropy secara natural bekerja dengan probabilitas, sehingga sangat cocok untuk output neural network yang diasumsikan merepresentasikan distribusi probabilitas kelas.
2. Penalti eksponensial untuk prediksi salah: Ketika prediksi sangat salah (misalnya memprediksi probabilitas mendekati 0 untuk kelas yang benar), cross entropy memberikan penalti yang sangat besar. Hal ini mendorong model untuk belajar memprediksi dengan lebih akurat.
3. Kombinabilitas dengan Softmax: Kombinasi cross entropy dengan fungsi aktivasi softmax sangat populer karena turunanannya yang sederhana dan komputasi yang efisien.
4. Interpretasi informasi: Secara konseptual, cross entropy mengukur jumlah bit informasi yang diperlukan untuk mengkodekan data jika menggunakan distribusi prediksi, dibandingkan dengan distribusi sebenarnya. Meminimalkan cross entropy setara dengan membuat distribusi prediksi sedekat mungkin dengan distribusi sebenarnya.

2.7 Inisialisasi Bobot

Weight initialization (inisialisasi bobot) merupakan langkah awal yang sangat penting dalam membangun neural network. Inisialisasi yang tepat dapat mempercepat konvergensi dan meningkatkan performa model, sedangkan inisialisasi yang buruk dapat menyebabkan masalah seperti vanishing/exploding gradients atau konvergensi yang sangat lambat.

2.7.1 Pentingnya Weight Initialization

Pada awal pelatihan, bobot neural network diinisialisasi dengan nilai tertentu. Nilai-nilai ini kemudian akan diperbarui melalui backpropagation untuk meminimalkan loss. Jika inisialisasi tidak tepat, beberapa masalah dapat muncul:

- Vanishing gradients: Jika bobot terlalu kecil, gradien yang propagate mundur akan semakin mengecil di setiap lapisan, menyebabkan lapisan-lapisan awal hampir tidak menerima pembaruan.
- Exploding gradients: Jika bobot terlalu besar, gradien dapat grow exponentially, menyebabkan pembaruan bobot yang tidak stabil dan oscillation.
- Symmetry problem: Jika semua bobot diinisialisasi dengan nilai yang sama, semua neuron dalam lapisan yang sama akan belajar hal yang sama, sehingga kapasitas jaringan tidak dimanfaatkan secara optimal.

2.7.2 Metode Inisialisasi yang Diuji

Implementasi inisialisasi bobot tersedia di modul src/initializers/ dengan metode-metode berikut:

2.7.2.1 Zero Initialization

Diimplementasikan di src/initializers/zero.py. Menginisialisasi semua bobot dengan nol menggunakan np.zeros(shape). Meskipun sederhana, metode ini tidak disarankan untuk jaringan yang lebih dalam karena menyebabkan semua neuron dalam lapisan tersembunyi memiliki identical computations, sehingga jaringan tidak dapat mempelajari fitur yang berbeda. Metode ini lebih sering digunakan sebagai baseline.

2.7.2.2 Uniform Initialization

Diimplementasikan di src/initializers/uniform.py. Menginisialisasi bobot dengan nilai acak dari distribusi uniform dalam interval. Default-nya adalah.

2.7.2.3 Normal Initialization

Diimplementasikan di src/initializers/normal.py. Menginisialisasi bobot dengan nilai acak dari distribusi normal dengan mean dan varians yang dapat dikonfigurasi. Default-nya adalah mean=0.0 dan variance=0.1.

2.7.2.4 Xavier Initialization

Diimplementasikan di src/initializers/bonus_initializers.py. Inisialisasi Xavier menyesuaikan skala bobot berdasarkan jumlah unit input dan output.

2.7.2.5 He Initialization

Diimplementasikan di src/initializers/bonus_initializers.py. Inisialisasi He menyesuaikan skala berdasarkan jumlah unit input.

2.7.3 Pengaruh terhadap Proses Training

Pemilihan metode inisialisasi mempengaruhi:

- Kecepatan konvergensi model
- Kemungkinan vanishing/exploding gradients
- Stabilitas proses pelatihan
- Kemampuan model untuk menemukan global minimum yang baik

Dalam tugas besar ini, kelima metode inisialisasi akan diuji dan dianalisis pengaruhnya terhadap performa model.

2.8 Regularisasi

Regularisasi merupakan teknik yang digunakan untuk mencegah overfitting (kondisi dimana model terlalu хорошо mempelajari data pelatihan tetapi gagal menggeneralisasi ke data baru). Overfitting terjadi ketika model memiliki kapasitas terlalu besar relatif terhadap kompleksitas data, sehingga model mulai menghafal noise dalam data pelatihan daripada mempelajari pola yang general.

2.8.1 Konsep Dasar Regularisasi

Regularisasi bekerja dengan menambahkan penalty term ke fungsi loss yang bergantung pada besarnya bobot. Dengan kata lain, model dihukum untuk memiliki bobot yang terlalu besar, mendorong model untuk mempelajari representasi yang lebih sederhana dan smooth. Regularisasi mengorbankan sedikit performa pada data pelatihan untuk meningkatkan generalisasi pada data baru.

2.8.2 L1 Regularization (Lasso)

Definisi: Menambahkan penalty proporsional dengan nilai absolut bobot ke fungsi loss.

$$L_{total} = L_{original} + \lambda \sum_i |w_i|$$

Karakteristik: L1 regularization cenderung menghasilkan solusi sparse, yaitu banyak bobot yang tepat menjadi nol. Hal ini secara efektif melakukan feature selection karena fitur-fitur dengan bobot nol dianggap tidak penting.

2.8.3 L2 Regularization (Ridge)

Definisi: Menambahkan penalty proporsional dengan kuadrat bobot ke fungsi loss.

$$L_{total} = L_{original} + \lambda \sum_i w_i^2$$

Karakteristik: L2 regularization cenderung menghasilkan semua bobot menjadi kecil tetapi tidak tepat nol. Metode ini lebih umum digunakan karena lebih stabil secara numerik dan sering memberikan performa yang lebih baik.

2.8.4 Parameter Lambda

Parameter λ (atau regularization strength) mengontrol seberapa besar penalty yang dikenakan pada bobot besar. Nilai λ yang terlalu kecil tidak akan memberikan efek regularisasi yang signifikan, sedangkan nilai λ yang terlalu besar dapat menyebabkan underfitting karena bobot menjadi terlalu mendekati nol dan model kehilangan kapasitas untuk mempelajari pola data.

2.9 Weight Update (Gradient Descent)

Pembaruan bobot dilakukan menggunakan algoritma gradient descent standar dengan persamaan:

Gradient Descent Update Rule

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \Delta \boldsymbol{w}$$

$$\Delta \boldsymbol{w} = -\eta \nabla E(\boldsymbol{w})$$

$$\Delta w_k = -\eta \frac{\partial E}{\partial w_k}$$

2.9.1 Gradient Descent Optimizer

Gradient Descent merupakan algoritma optimasi fundamental yang digunakan untuk memperbarui bobot neural network berdasarkan gradien yang dihitung melalui backpropagation. Prinsip dasar gradient descent adalah bergerak ke arah yang mengurangi fungsi loss secara paling curam. Implementasi optimizer tersedia di src/optimizers/.

2.9.2 Peran Learning Rate

Learning rate merupakan *hyperparameter* paling penting yang mengontrol ukuran langkah dalam proses optimasi:

- Learning rate terlalu besar: Dapat menyebabkan oscillation atau divergence (loss meningkat)
- Learning rate terlalu kecil: Menyebabkan konvergensi yang sangat lambat
- Learning rate yang tepat: Menghasilkan konvergensi yang cepat dan stabil

Implementasi FFNN mendukung kedua optimizer ini melalui parameter optimizer pada method train().

2.10 Automatic Differentiation

Sebagai variasi implementasi, tugas besar ini juga menyediakan implementasi Automatic Differentiation (autodiff) yang dapat digunakan sebagai alternatif untuk backpropagation manual. Implementasi ini terinspirasi dari library micrograd dan menyediakan cara yang lebih intuitif untuk melakukan diferensiasi otomatis.

2.10.1 Kelas Value

Implementasi utama berada di src/autodiff/value.py. Kelas Value merepresentasikan nilai skalar yang dapat melacak operasi matematika dan secara otomatis menghitung gradien menggunakan backpropagation.

Setiap objek Value menyimpan:

- data: Nilai skalar aktual
- grad: Gradien terhadap fungsi loss
- _prev: Himpunan nilai anak (operand yang membuat nilai ini)
- _op: Operasi yang membuat nilai ini
- _backward: Fungsi untuk menghitung gradien lokal

2.10.2 Overloading Operator

Kelas Value mengimplementasikan operasi matematika melalui Python magic methods:

- __add__, __radd__: Penjumlahan dengan gradien
- __sub__, __rsub__: Pengurangan
- __mul__, __rmul__: Perkalian dengan gradien
- __truediv__, __rtruediv__: Pembagian
- __pow__: Perpangkatan dengan gradien

2.10.3 Fungsi Aktivasi

Kelas Value juga mengimplementasikan fungsi aktivasi: relu(), sigmoid(), tanh(), exp()

2.10.4 Computational Graph dan Topological Sort

Ketika operasi dilakukan pada objek Value, sebuah computational graph dibangun secara dinamis. Setiap operasi menyimpan referensi ke operand-nya (children) dan operasi yang dilakukan.

Method backward() melakukan backpropagation menggunakan topological sort:

1. Bangun urutan topologis dari semua node dalam graph menggunakan DFS
2. Set gradien awal = 1.0 untuk node output
3. Iterate node dalam urutan terbalik (dari output ke input)
4. Panggil _backward() pada setiap node untuk propagasi gradien

2.10.5 Integrasi dengan FFNN

Implementasi autodiff dapat digunakan untuk membangun FFNN dengan cara yang lebih deklaratif. Setiap operasi matematika (perkalian matriks, penjumlahan, aktivasi) secara otomatis membangun graph dan menghitung gradien ketika backward() dipanggil.

2.11 Adam Optimizer

Diimplementasikan di src/optimizers/adam.py. Adam (Adaptive Moment Estimation) menggabungkan manfaat dari momentum dan RMSProp:

1. First Moment (Momentum):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g$$

2. Second Moment (RMSProp-like):

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g^2$$

3. Bias Correction:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

4. Update:

$$w_t = w_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

2.12 RMSNorm

RMSNorm (Root Mean Square Normalization) merupakan teknik normalisasi yang lebih sederhana dibandingkan LayerNormalization namun tetap efektif dalam menstabilkan pelatihan neural network. Berbeda dengan LayerNormalization yang menormalisasi berdasarkan mean dan varians, RMSNorm hanya menggunakan root mean square (RMS)

2.12.1 Implementasi Forward Pass

Implementasi RMSNorm dapat dilihat di src/normalization/rmsnorm.py. Pada forward pass:

```
def forward(self, x, training=True):
    self.last_input = x

    mean_square = np.mean(x ** 2, axis=-1, keepdims=True)
    rms = np.sqrt(mean_square + self.epsilon)

    output = self.gain * (x / rms)
    return output
```

2.12.2 Parameter Gain

Berbeda dengan LayerNormalization yang memiliki parameter γ (scale) dan β (shift), RMSNorm hanya memiliki parameter γ (gain) yang diinisialisasi ke 1.0. Parameter ini dapat dipelajari selama pelatihan dan memungkinkan model untuk mempelajari skala output yang optimal.

2.12.3 Backward Pass

Pada backward pass, gradien dihitung menggunakan chain rule:

1. Gradien terhadap gain:

$$\frac{\partial L}{\partial \gamma} = \sum \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{\partial \hat{x}_i}{\partial \gamma} = \sum \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{x_i}{\mathrm{RMS}(x)}$$

2. Gradien terhadap input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \hat{x}} \cdot \gamma \cdot \left(\frac{1}{\mathrm{RMS}(x)} - \frac{x \cdot x}{\mathrm{RMS}(x)^3 \cdot d} \right)$$

2.12.4 Keunggulan RMSNorm

1. Kompleksitas komputasi lebih rendah: Tidak perlu menghitung mean
2. lebih sedikit parameter: Hanya γ, tidak ada β
3. Efektif untuk transformer: RMSNorm lebih cepat dan bekerja dengan baik dalam arsitektur transformer
4. Invarian terhadap translasi: Tidak sensitif terhadap pergeseran mean

2.12.5 Integrasi dengan FFNN

RMSNorm dapat diintegrasikan ke dalam FFNN melalui parameter normalization pada konstruktor:

```
model = FFNN(
    layer_sizes=[64, 32, 16, 1],
    activations=['relu', 'relu', 'sigmoid'],
    normalization=['rmsnorm', 'rmsnorm', None]
)
```

Pada forward pass, RMSNorm diterapkan sebelum fungsi aktivasi. Pada backward pass, gradien di-propagate melalui layer normalisasi.

3. Pengujian

3.1 Setup Eksperimen

Semua eksperimen dilakukan menggunakan dataset datasetml_2026.csv dengan target prediksi placement_status. Dataset terdiri dari 10.000 sampel dengan 12 fitur yang mencakup data numerik dan kategorikal. Setelah preprocessing (one-hot encoding fitur kategorikal, feature engineering interaksi, dan StandardScaler), total fitur yang digunakan berjumlah sekitar 30+ fitur.

Pembagian data menggunakan strategi train/val/test dengan rasio 64%/16%/20% (stratified split). Arsitektur dasar yang digunakan sebagai baseline adalah [n_features, 32, 16, n_classes] dengan aktivasi ReLU, loss Categorical Cross-Entropy, initializer Uniform, dan learning rate 0.01 (kecuali disebutkan lain). Semua eksperimen dijalankan selama maksimal 50 epoch dengan early stopping berbasis validation loss (patience=20).

3.2 Evaluation Metrics

Metrik evaluasi yang digunakan pada seluruh eksperimen adalah:

- Accuracy: proporsi prediksi benar dari total prediksi.
- Train Loss & Validation Loss: nilai categorical cross-entropy pada set latih dan validasi per epoch.
- Precision: TP / (TP + FP) yakni akurasi prediksi positif.
- Recall: TP / (TP + FN) yakni kemampuan mendeteksi semua positif.
- F1-Score: rata-rata harmonik precision dan recall.

3.3 Pengaruh Depth dan Width

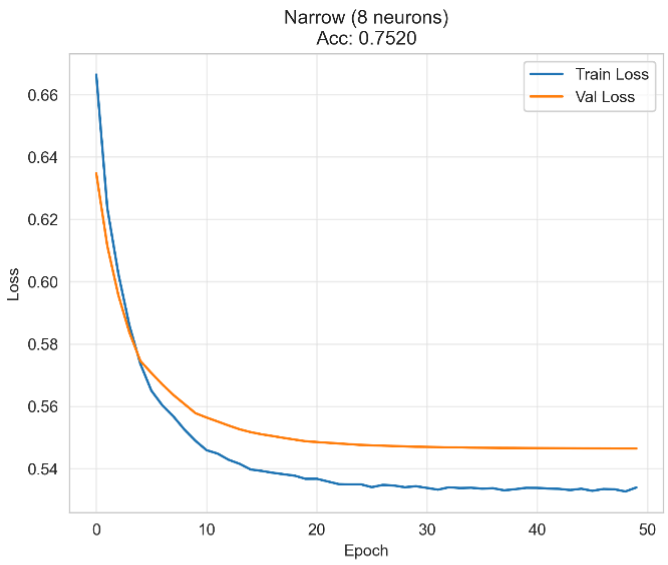
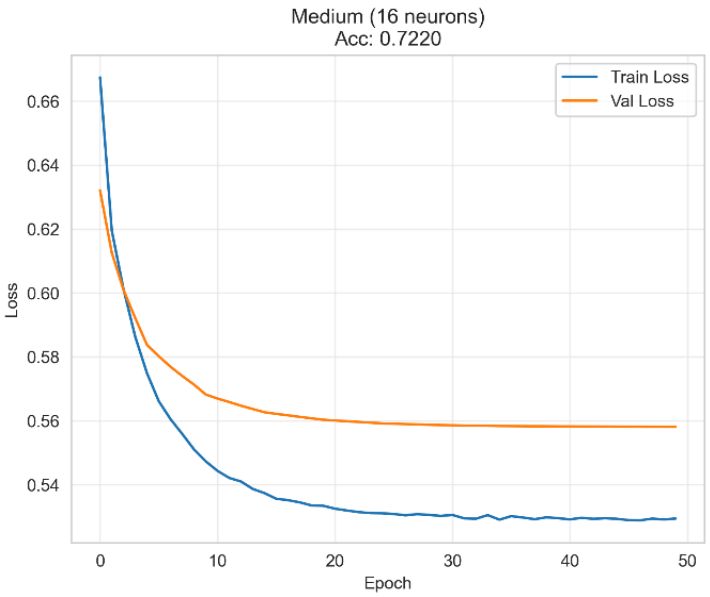
Eksperimen ini bertujuan menganalisis pengaruh kedalaman (depth) dan lebar (width) jaringan terhadap performa model. Hyperparameter lain dibuat tetap: aktivasi ReLU, learning rate 0.01, batch size 32, epochs 50.

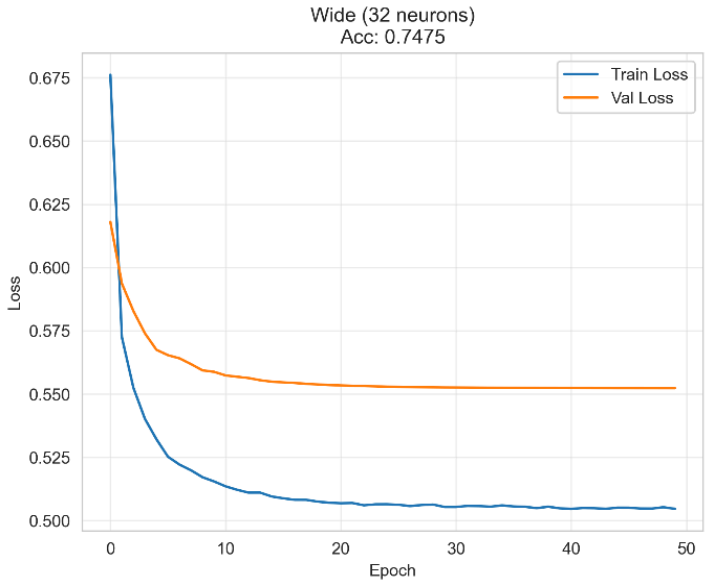
3.3.1 Variasi Width (Depth Tetap)

Depth tetap: 2 hidden layers. Width yang diuji: 8 neuron (Narrow), 16 neuron (Medium), dan 32 neuron (Wide) per hidden layer.

Arsitektur	Train Loss	Val Loss	Accuracy	Keterangan
Narrow (8 neurons) [n_feat, 8, 8, 2]	0.5340	0.5465	75.20%	Kapasitas rendah, konvergensi lambat karena bottleneck
Medium (16 neurons) [n_feat, 16, 16, 2]	0.5294	0.5581	72.20%	Performa terendah; kemungkinan tidak

Arsitektur	Train Loss	Val Loss	Accuracy	Keterangan
				konvergen optimal dalam 50 epoch
Wide (32 neurons) [n_feat, 32, 32, 2]	0.5046	0.5523	74.75%	Train loss terbaik; gap train-val mengindikasikan slight overfitting

Arsitektur	Grafik Training loss dan validation loss
Narrow (8 neurons) [n_feat, 8, 8, 2]	
Medium (16 neurons) [n_feat, 16, 16, 2]	

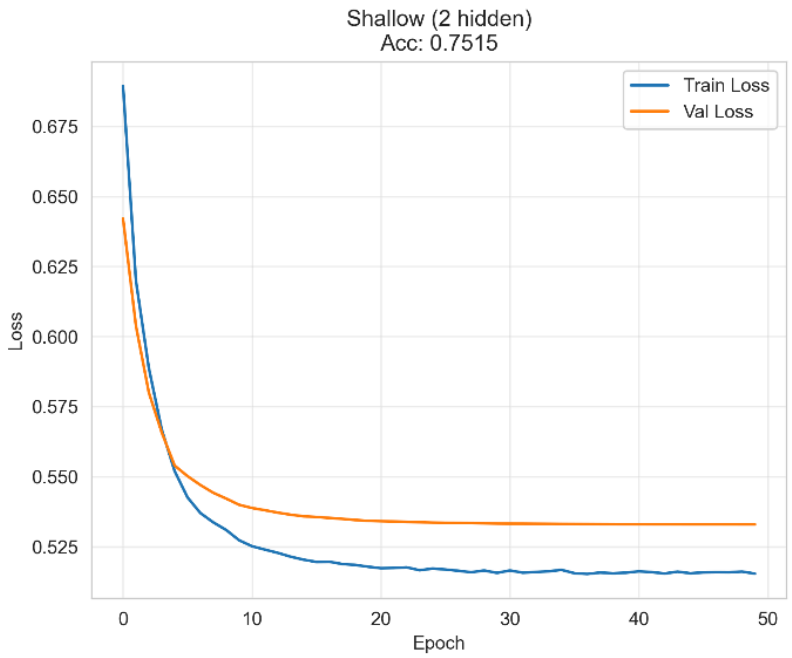
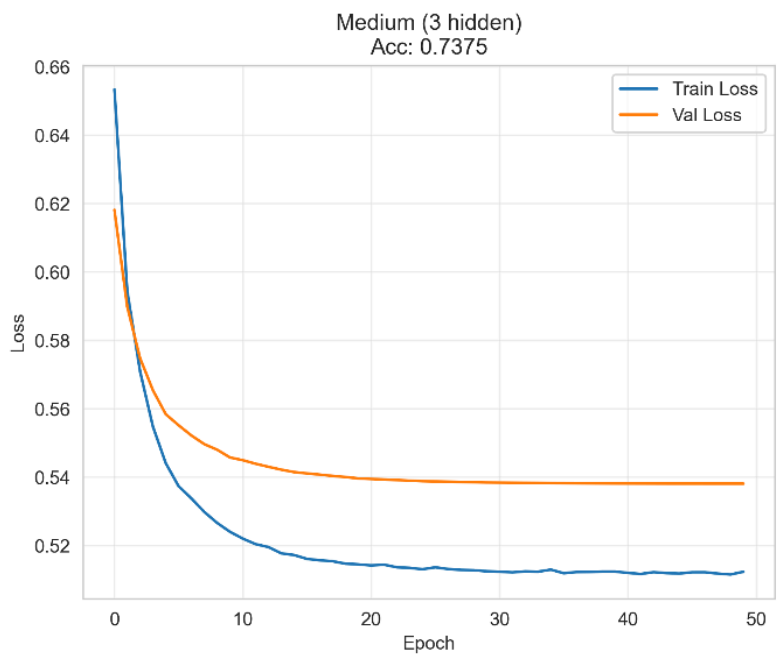
Wide (32 neurons) [n_feat, 32, 32, 2]	

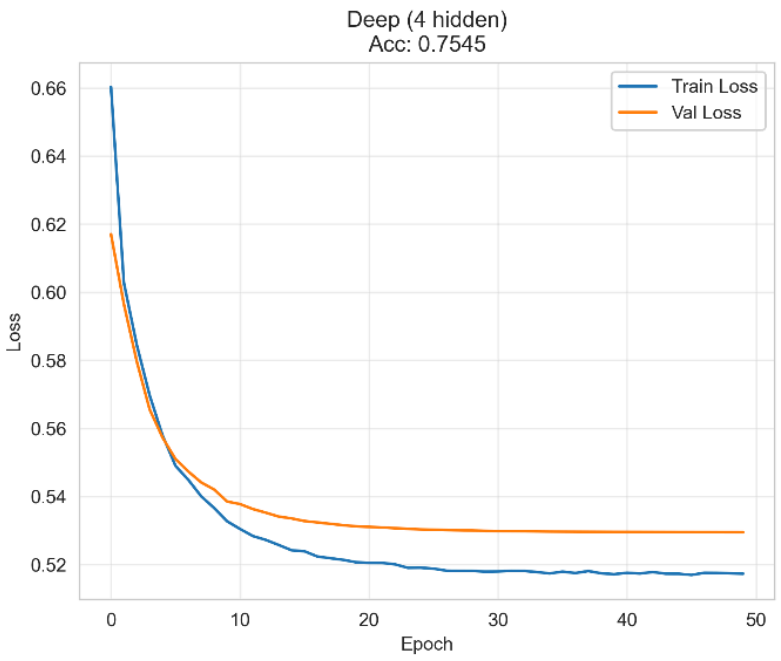
Analisis: Hasil menunjukkan bahwa width 8 neuron (Narrow) justru memberikan accuracy terbaik (75.20%), disusul Wide (74.75%) dan Medium (72.20%). Fenomena ini mengindikasikan bahwa pada dataset placement ini, kapasitas jaringan yang terlalu besar (16 neuron per layer) dapat menyebabkan kesulitan optimasi dalam jumlah epoch yang terbatas. Model Wide memiliki train loss terendah (0.5046) namun gap dengan val loss lebih besar, mengindikasikan potensi overfitting. Model Narrow dengan kapasitas terbatas cenderung lebih stabil dan menghasilkan generalisasi yang lebih baik pada kasus ini.

3.3.2 Variasi Depth (Width Tetap)

Width tetap: 16 neuron per hidden layer. Depth yang diuji: 2 hidden layers (Shallow), 3 hidden layers (Medium), dan 4 hidden layers (Deep).

Arsitektur	Train Loss	Val Loss	Accuracy	Keterangan
Shallow (2 hidden) [n_feat,16,16,2]	0.5154	0.5330	75.15%	Konvergensi cepat dan stabil, gap kecil menunjukkan generalisasi baik
Medium (3 hidden) [n_feat,16,16,16,2]	0.5123	0.5381	73.75%	Train loss sedikit lebih baik namun accuracy menurun
Deep (4 hidden) [n_feat,16,16,16,16,2]	0.5173	0.5295	75.45%	Val loss terbaik; depth tambahan membantu representasi kompleks

Arsitektur	Grafik training loss dan validation loss
Shallow (2 hidden) [n_feat,16,16,2]	
Medium (3 hidden) [n_feat,16,16,16,2]	

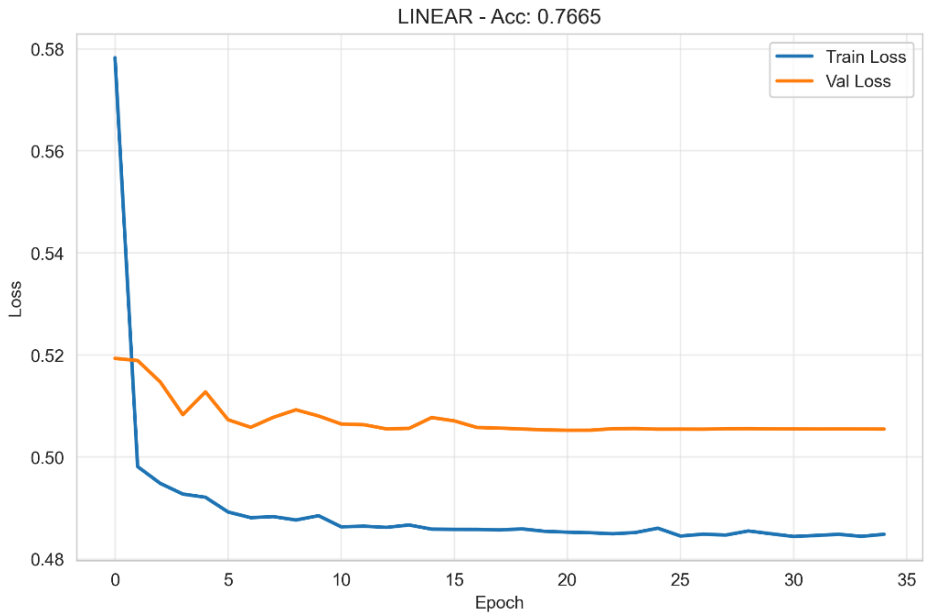
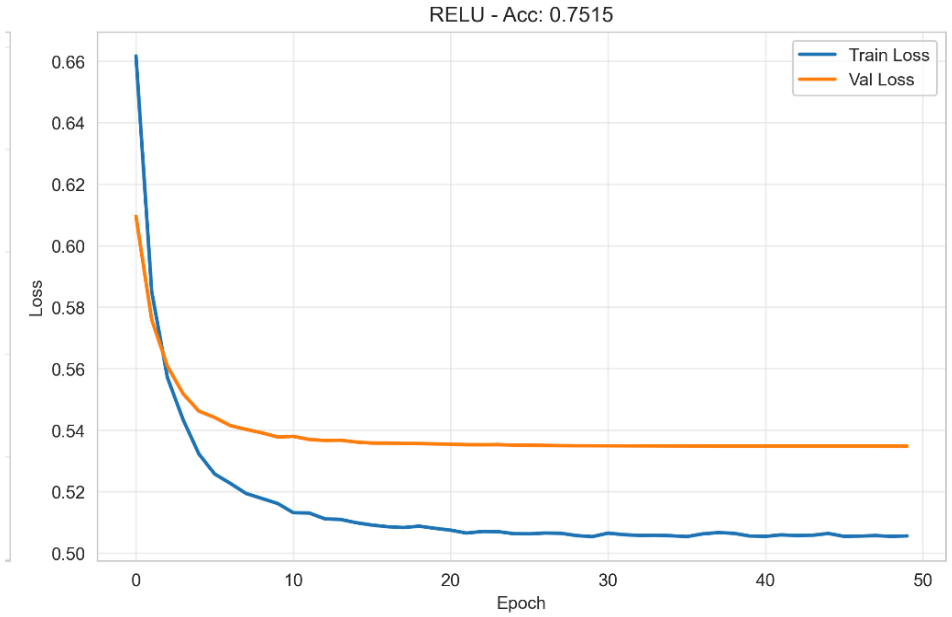
Deep (4 hidden) [n_feat,16,16,16,16,2]	

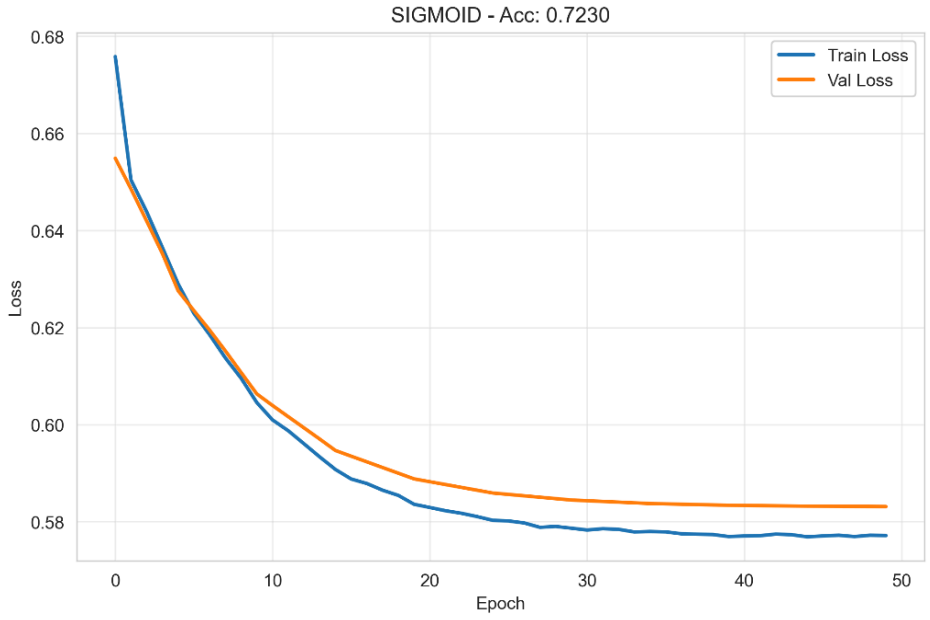
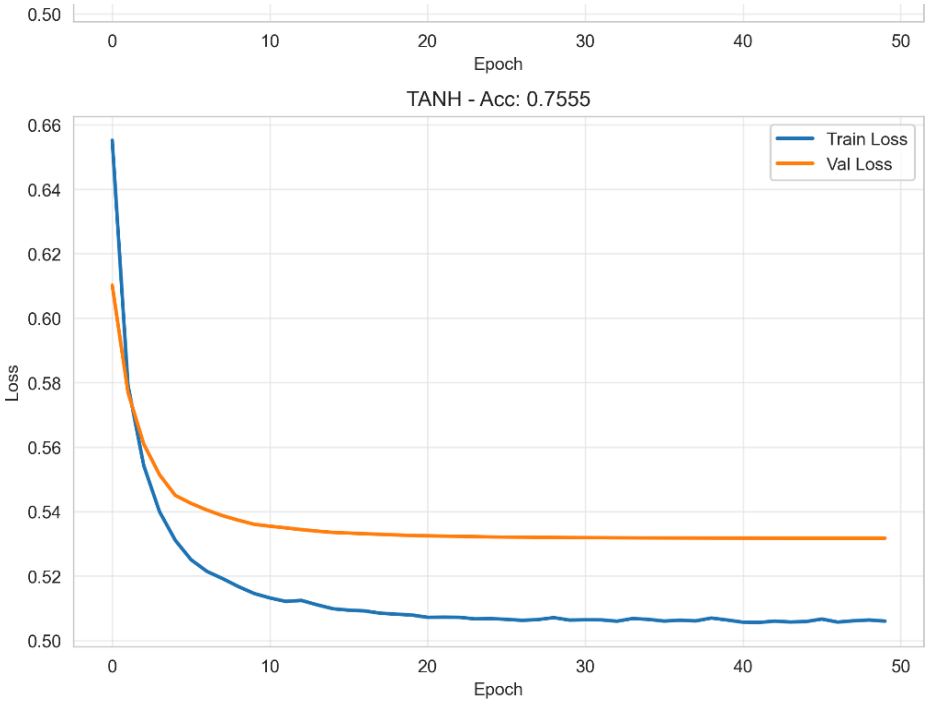
Analisis: Secara keseluruhan, penambahan depth memberikan pengaruh yang relatif kecil pada dataset ini. Model Deep (4 hidden) mencapai accuracy tertinggi (75.45%) dan val loss terbaik (0.5295), menunjukkan bahwa representasi yang lebih dalam membantu sedikit dalam menangkap pola non-linear. Namun perbedaannya tidak signifikan (~0.3% dibanding Shallow). Risiko vanishing gradient belum terlihat jelas pada depth 4 layer dengan aktivasi ReLU. Model Shallow tetap kompetitif dengan konvergensi yang lebih cepat dan stabil.

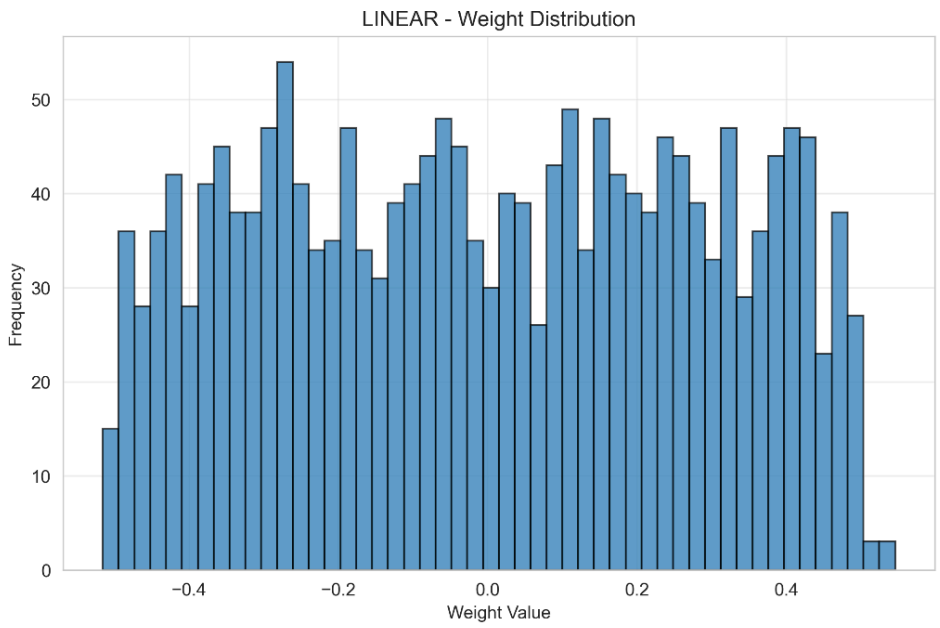
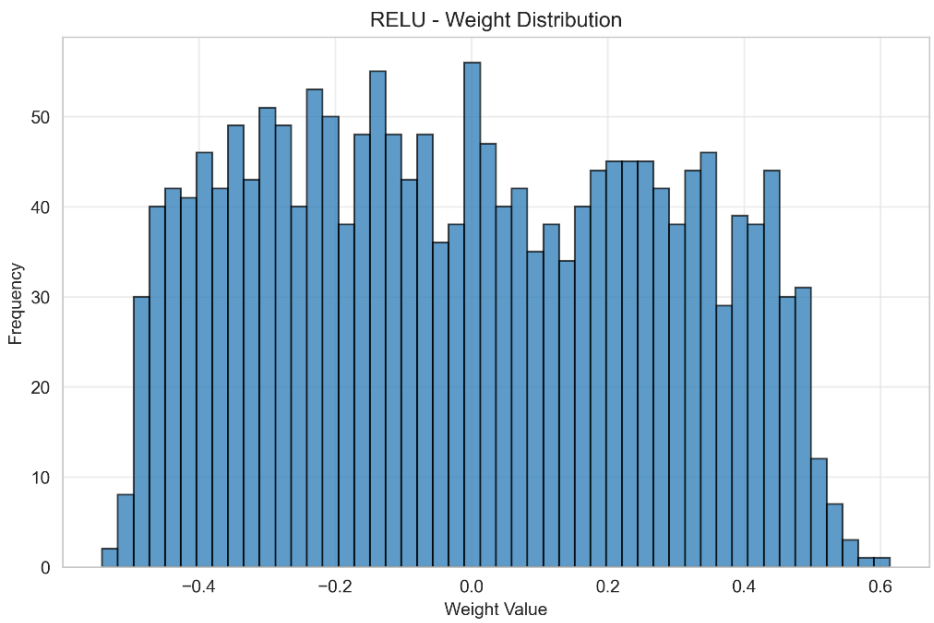
3.4 Pengaruh Fungsi Aktivasi

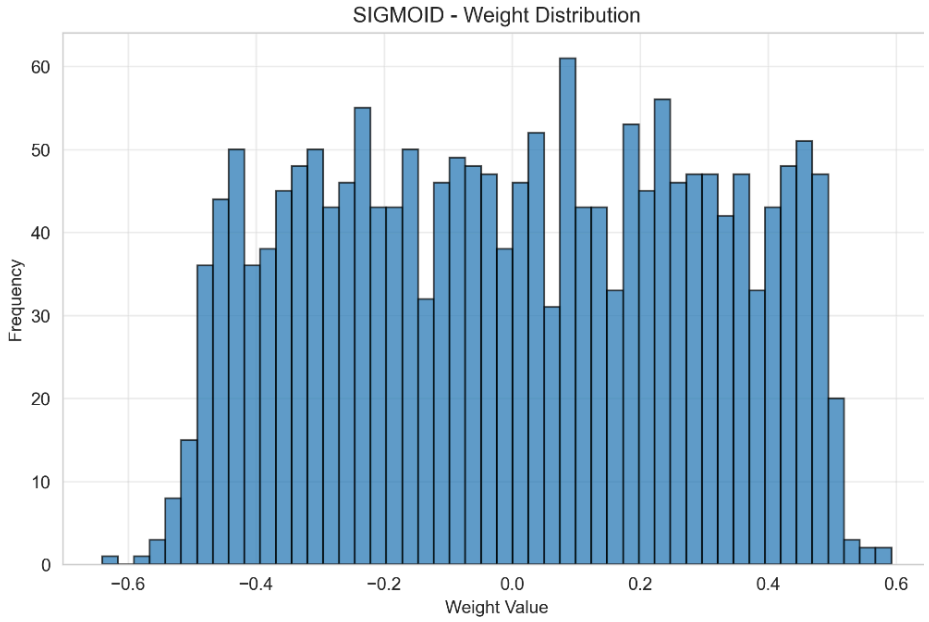
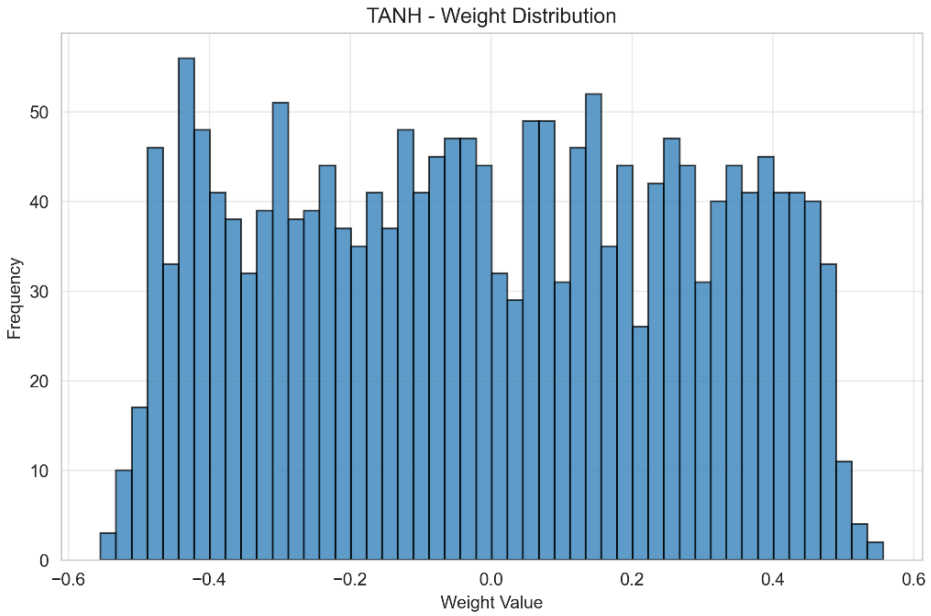
Arsitektur dasar yang digunakan: [n_features, 32, 16, n_classes] (3 layer total, 2 hidden). Layer output menggunakan Softmax. Fungsi aktivasi yang diuji pada hidden layers: Linear, ReLU, Sigmoid, dan Tanh.

Aktivasi	Train Loss	Val Loss	Accuracy	Keterangan
Linear	0.4848	0.5054	76.65%	Accuracy tertinggi; efektif karena dataset cenderung separabel secara linear setelah feature engineering
ReLU	0.5056	0.5348	75.15%	Konvergensi stabil; mencegah vanishing gradient; performa baik secara keseluruhan
Sigmoid	0.5772	0.5831	72.30%	Performa terburuk; rentan vanishing gradient, konvergensi lambat
Tanh	0.5060	0.5317	75.55%	Lebih baik dari Sigmoid; zero-centered membantu optimasi, mendekati ReLU

Aktivasi	Grafik training loss dan validation loss
Linear	
ReLU	

Sigmoid	
Tanh	

Aktivasi	Grafik weight distribution
Linear	
ReLU	

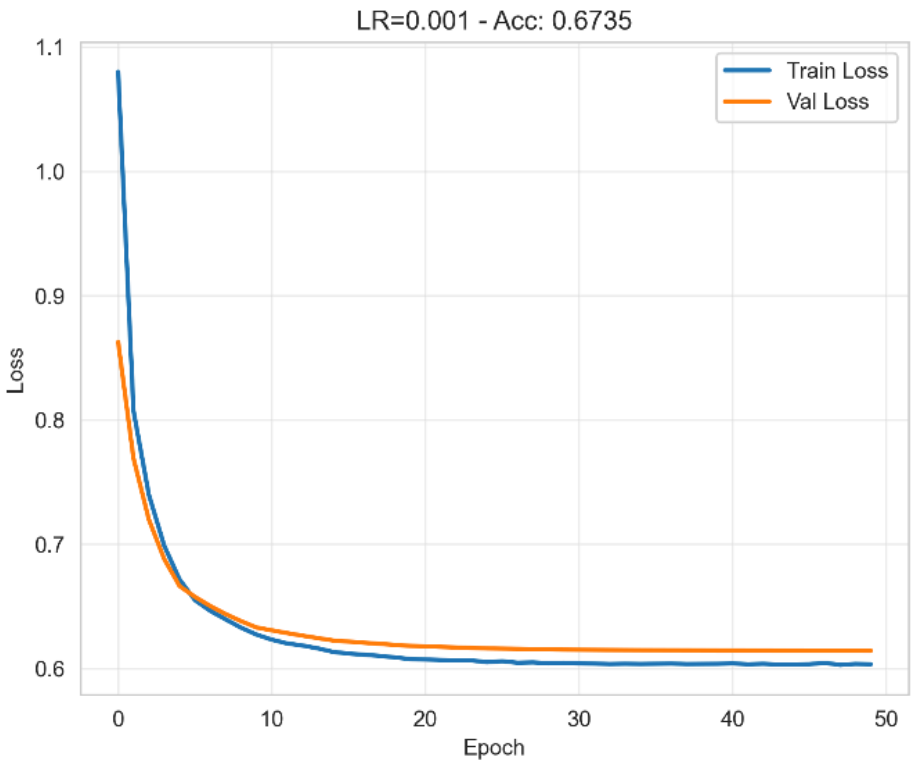
Sigmoid	
Tanh	

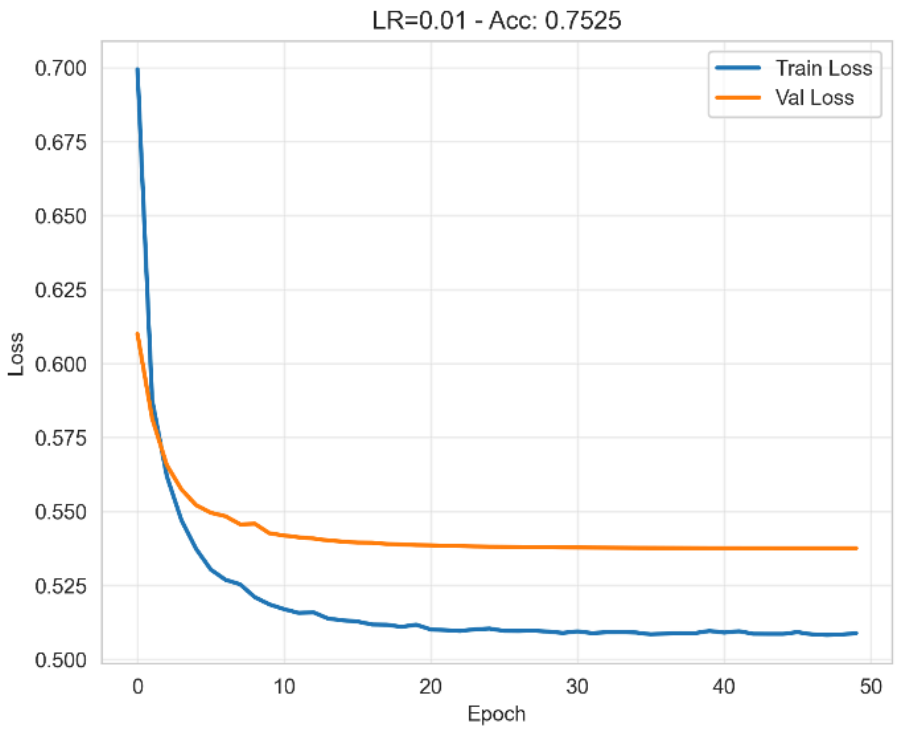
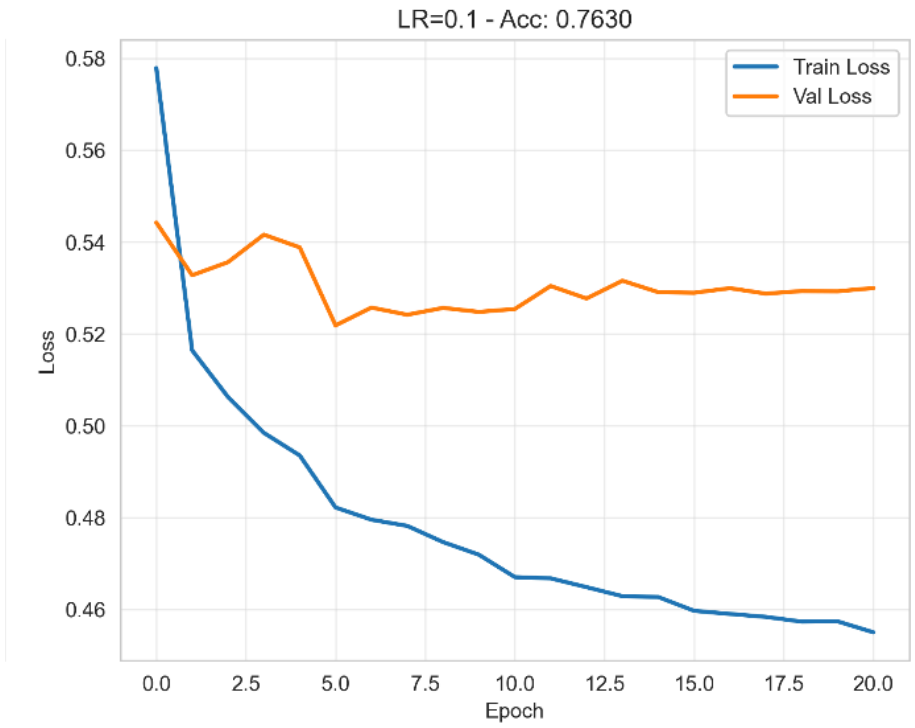
Analisis: Aktivasi Linear memberikan accuracy terbaik (76.65%) yang mengindikasikan bahwa setelah feature engineering yang kaya (30+ fitur termasuk interaksi), hubungan antara fitur dan target placement_status cenderung linear. Sigmoid memiliki performa terburuk karena saturasi di kedua ujung (nilai mendekati 0 atau 1) menyebabkan vanishing gradient, terlihat dari train loss yang tinggi (0.5772) dan konvergensi yang sangat lambat. Tanh lebih baik dari Sigmoid karena zero-centered sehingga gradien lebih seimbang. ReLU menjadi pilihan praktis terbaik karena komputasi ringan, stabil, dan tidak mengalami saturasi pada nilai positif.

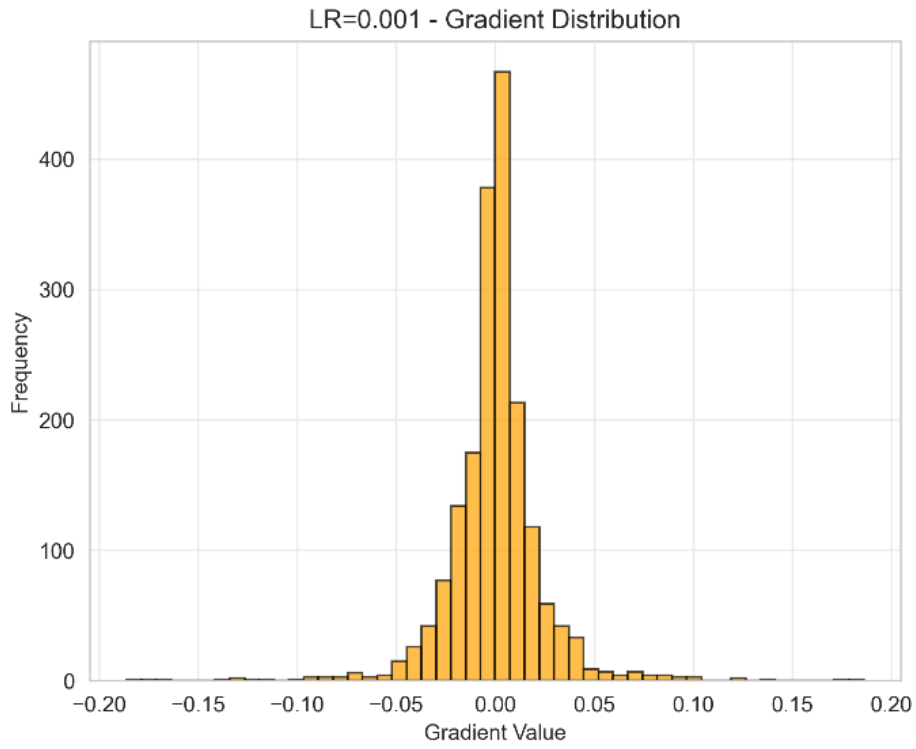
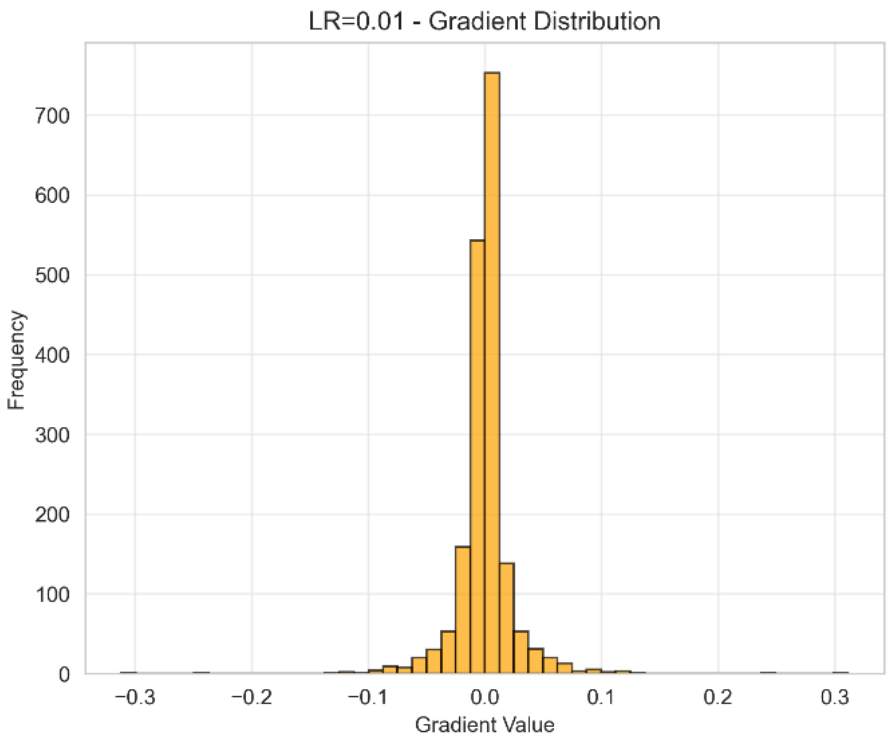
3.5 Pengaruh Learning Rate

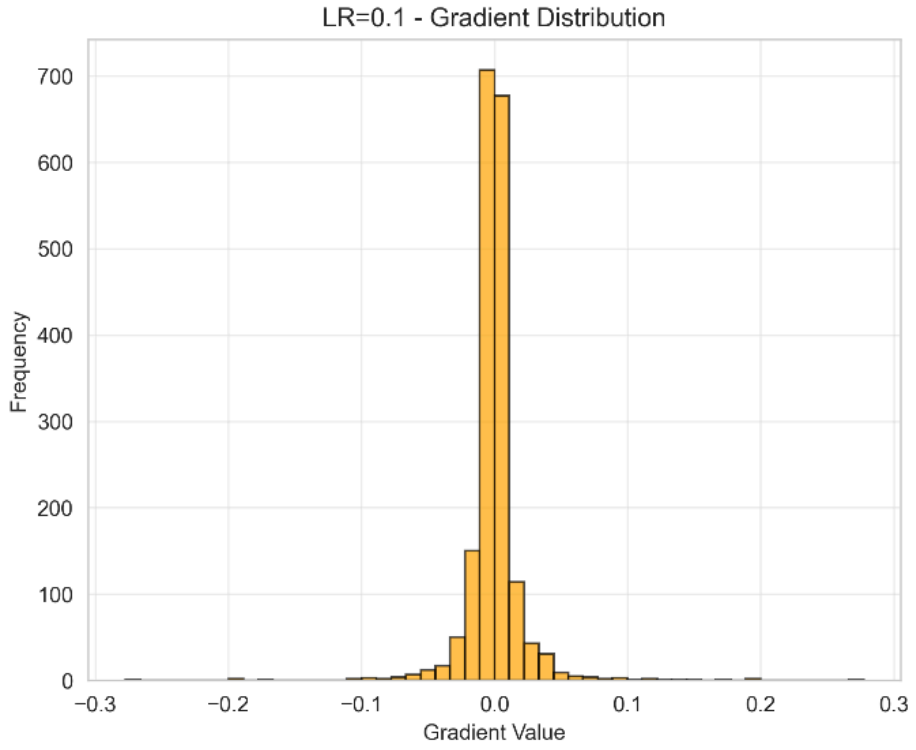
Arsitektur: **[n_features, 32, 16, n_classes]**, aktivasi ReLU, batch size 32, epochs 50. Tiga variasi learning rate diuji: 0.001, 0.01, dan 0.1.

Learning Rate	**Train Loss**	**Val Loss**	**Accuracy**	**Keterangan**
LR = 0.001	0.6034	0.6143	67.35%	Terlalu kecil; konvergensi sangat lambat, 50 epoch belum cukup untuk konvergen
LR = 0.01	0.5089	0.5376	75.25%	Optimal; konvergensi stabil dan cepat dengan generalisasi baik
LR = 0.1	0.4550	0.5300	76.30%	Terbesar; train loss terendah namun rentan osilasi di awal training

Learning Rate	Grafik training loss dan validation loss
LR=0,001	

LR=0,01	 LR=0.01 - Acc: 0.7525 Train Loss Val Loss Loss 0.700 0.675 0.650 0.625 0.600 0.575 0.550 0.525 0.500 0 10 20 30 40 50 Epoch
LR=0,1	 LR=0.1 - Acc: 0.7630 Train Loss Val Loss Loss 0.58 0.56 0.54 0.52 0.50 0.48 0.46 0.0 2.5 5.0 7.5 10.0 12.5 15.0 17.5 20.0 Epoch

Learning Rate	Grafik gradient Distribution
LR=0,001	
LR=0,01	

LR = 0.1	

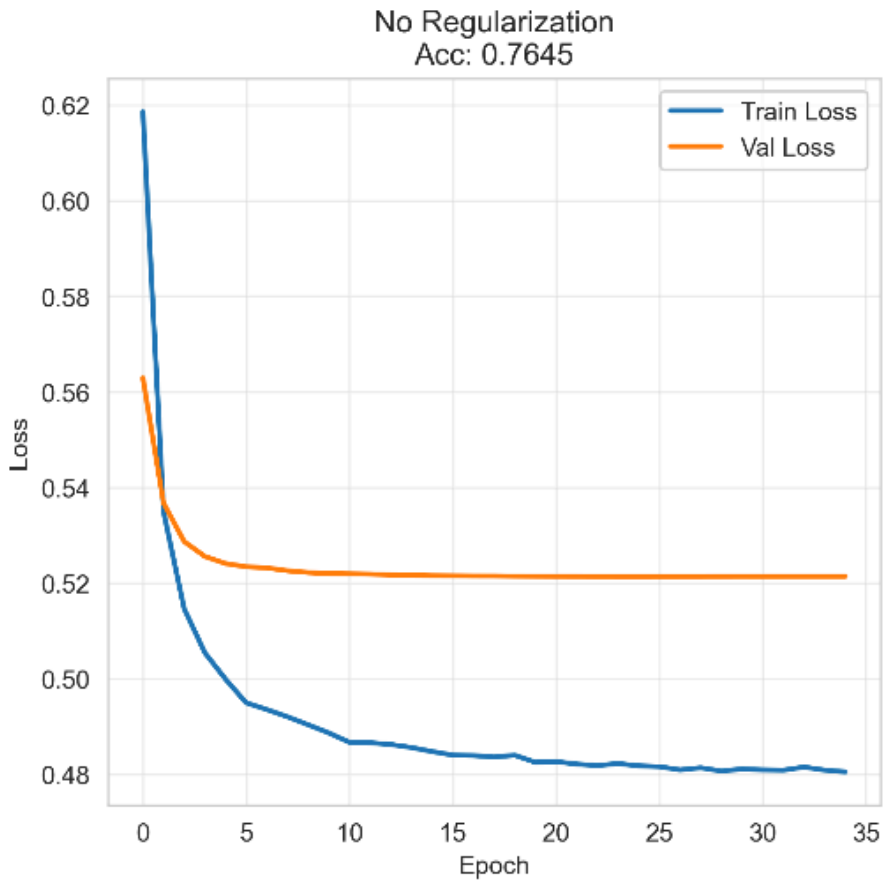
Analisis: Learning rate 0.1 menghasilkan accuracy tertinggi (76.30%) dan train loss terbaik (0.4550), namun dengan risiko osilasi pada awal training yang terlihat dari kurva loss yang tidak stabil di epoch pertama. Learning rate 0.001 menunjukkan gejala konvergensi lambat yang jelas, setelah 50 epoch, loss masih berada di 0.6034 yang jauh dari konvergen. Learning rate 0.01 menjadi pilihan optimal yang menyeimbangkan kecepatan konvergensi dan stabilitas. Penggunaan LR scheduler (Reduce on Plateau) pada eksperimen ini membantu semua konfigurasi beradaptasi, sehingga LR 0.1 tidak diverge meskipun nilainya besar.

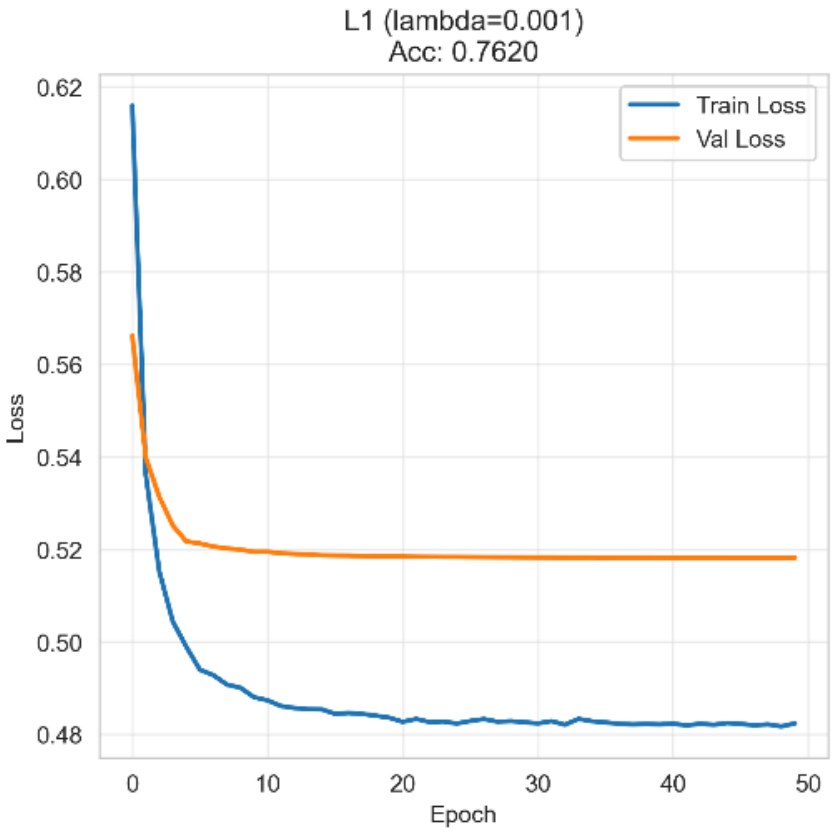
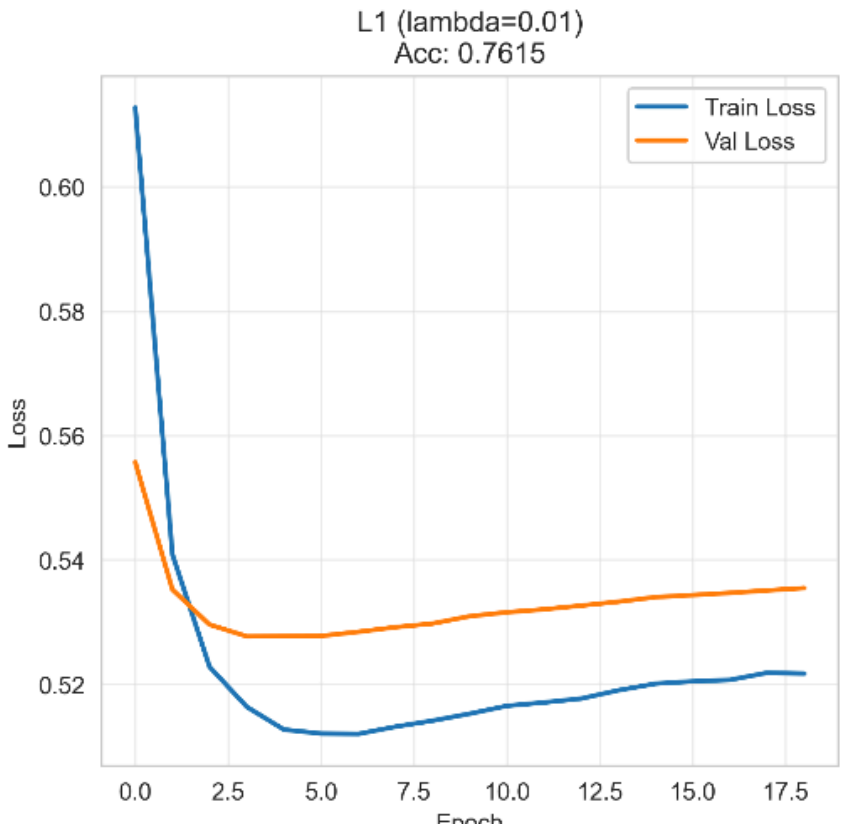
3.6 Pengaruh Regularisasi

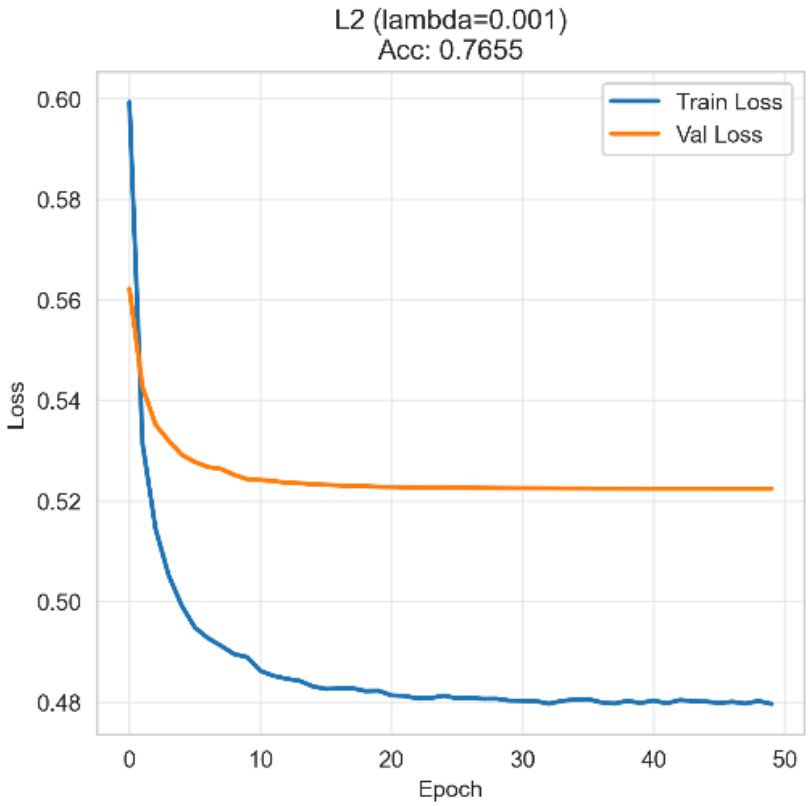
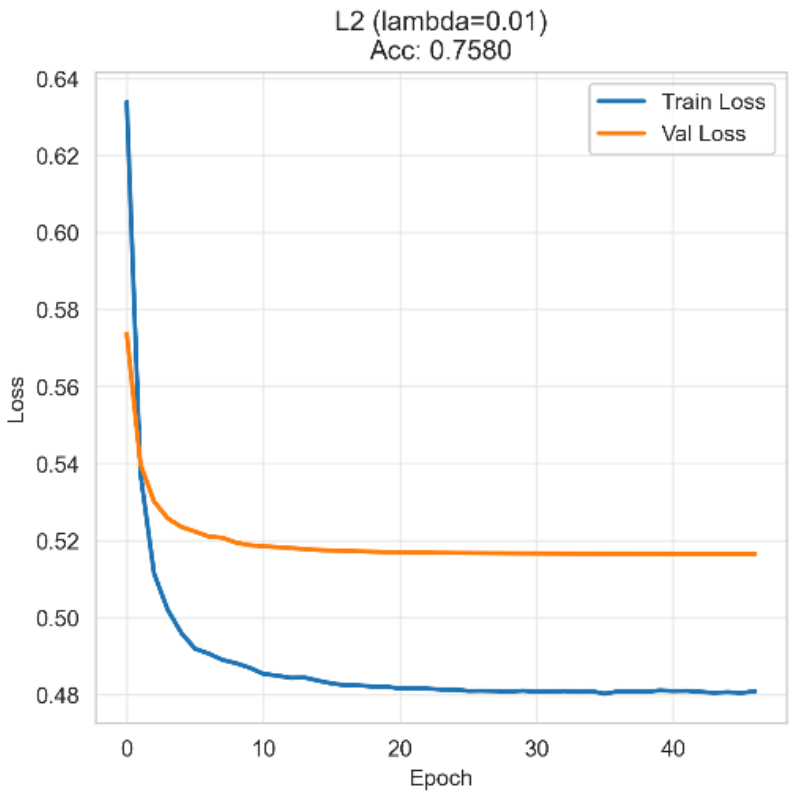
Arsitektur: **[n_features, 32, 16, n_classes]**, aktivasi ReLU, learning rate 0.01, batch size 32, epochs 50. Nilai lambda yang diuji: 0.001 dan 0.01 untuk L1 dan L2. Dropout rates yang diuji: 0.2 dan 0.5.

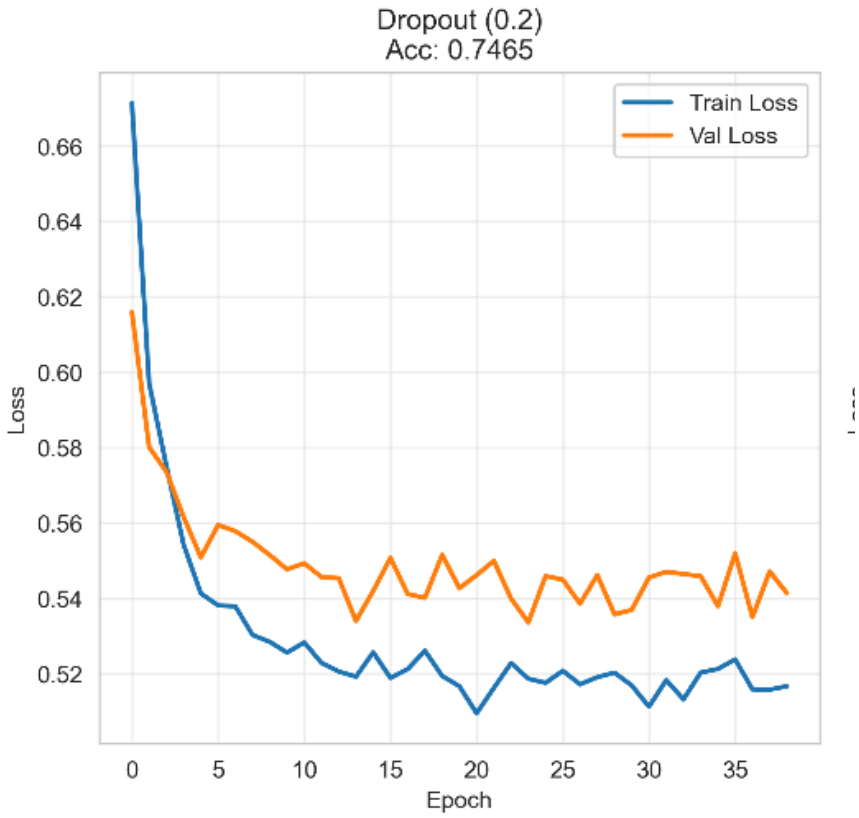
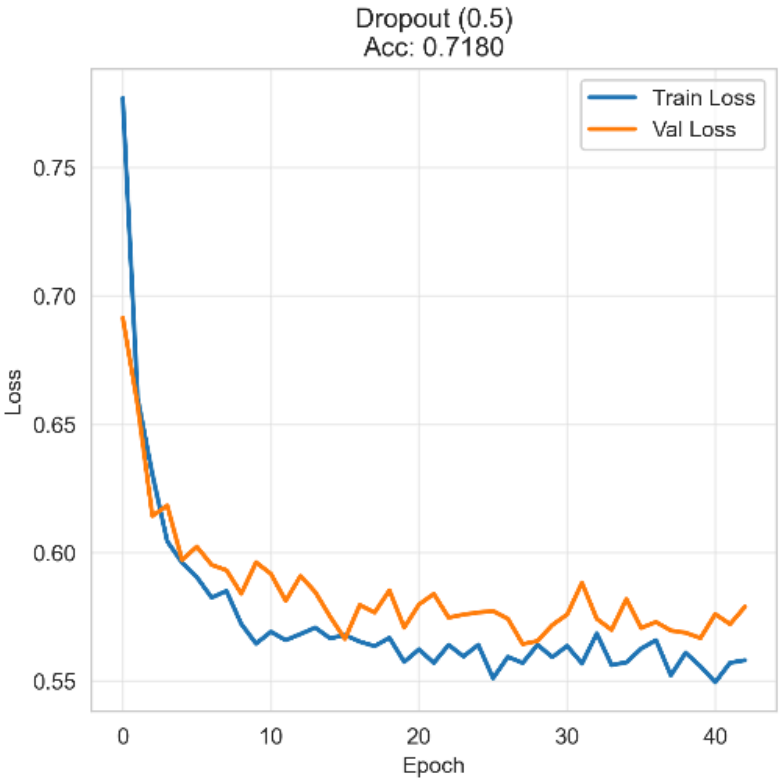
Regularisasi	Train Loss	Val Loss	Accuracy	Keterangan
Tanpa Regularisasi	0.4805	0.5214	76.45%	Baseline; performa baik namun rentan overfitting
L1 (lambda=0.001)	0.4824	0.5182	76.20%	Val loss sedikit lebih baik dari baseline; mendorong sparsity bobot
L1 (lambda=0.01)	0.5217	0.5355	76.15%	Lambda terlalu besar; penalti terlalu kuat menyebabkan underfitting
L2 (lambda=0.001)	0.4796	0.5224	76.55%	Accuracy terbaik; bobot kecil merata mencegah dominasi fitur tertentu

Regularisasi	Train Loss	Val Loss	Accuracy	Keterangan
L2 (lambda=0.01)	0.4808	0.5165	75.80%	Val loss terbaik (0.5165) namun accuracy sedikit turun
Dropout (rate=0.2)	0.5167	0.5414	74.65%	Regularisasi kuat; dropout mempersulit konvergensi
Dropout (rate=0.5)	0.5579	0.5787	71.80%	Terlalu agresif; menghilangkan terlalu banyak informasi per batch

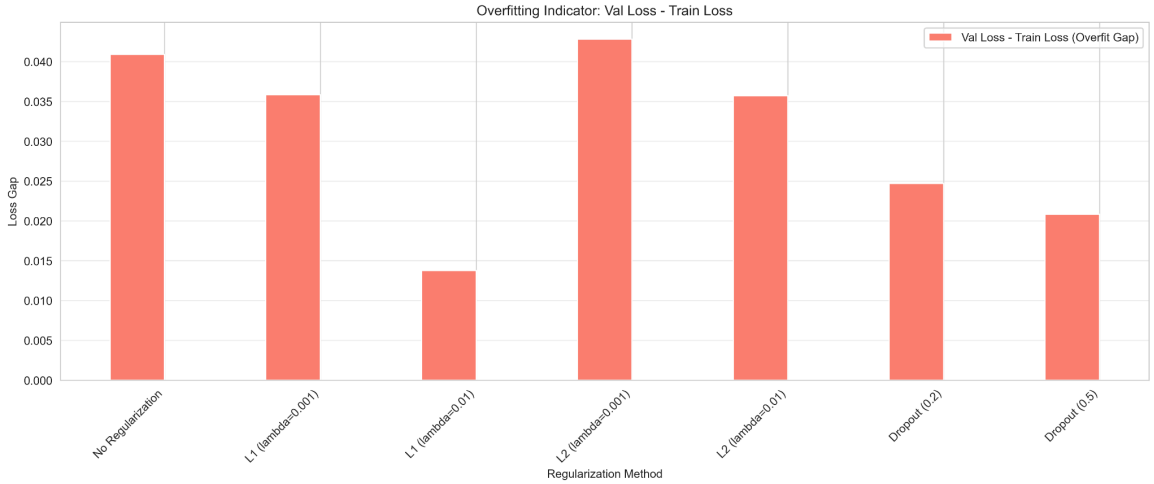
Regularisasi	Grafik training loss dan validation loss
Tanpa Regularisasi	

L1 (lambda=0.001)	
L1 (lambda=0.01)	

L2 (lambda=0.001)	
L2 (lambda=0.01)	

Dropout (rate=0.2)	
Dropout (rate=0.5)	

Grafik Overfitting indicator:



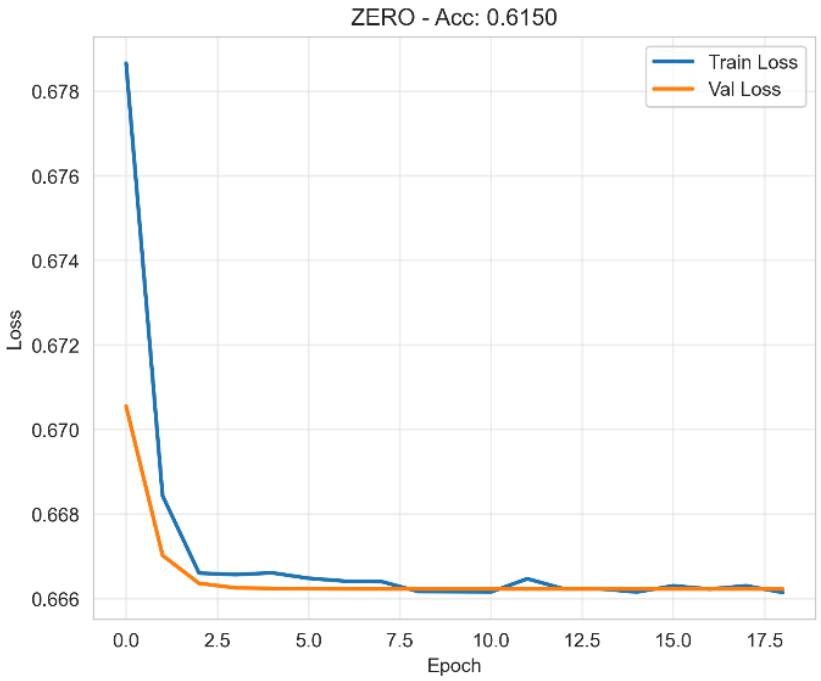
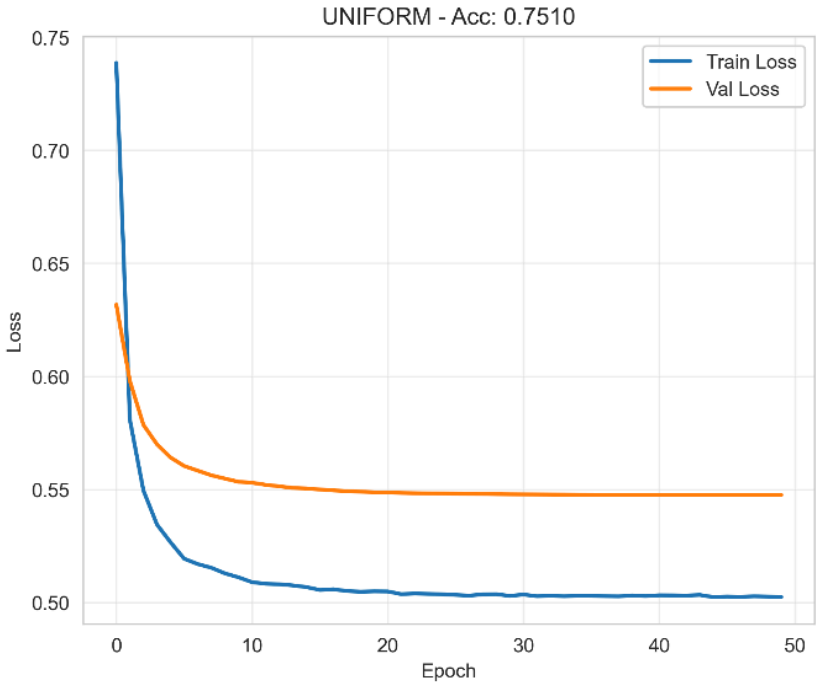
Analisis: L2 dengan lambda=0.001 memberikan accuracy terbaik (76.55%) dan menghasilkan bobot yang lebih kecil secara merata, mencegah bobot terlalu besar tanpa menghapus fitur sepenuhnya. L1 dengan lambda=0.001 memberikan val loss lebih rendah (0.5182 vs 0.5214 baseline), menunjukkan regularisasi ringan membantu generalisasi. Lambda yang terlalu besar (0.01) untuk L1 menyebabkan penalti berlebihan sehingga train loss meningkat signifikan. Dropout yang agresif (0.5) memberikan performa terburuk karena terlalu banyak informasi dihilangkan per forward pass, menghambat model belajar pola yang konsisten.

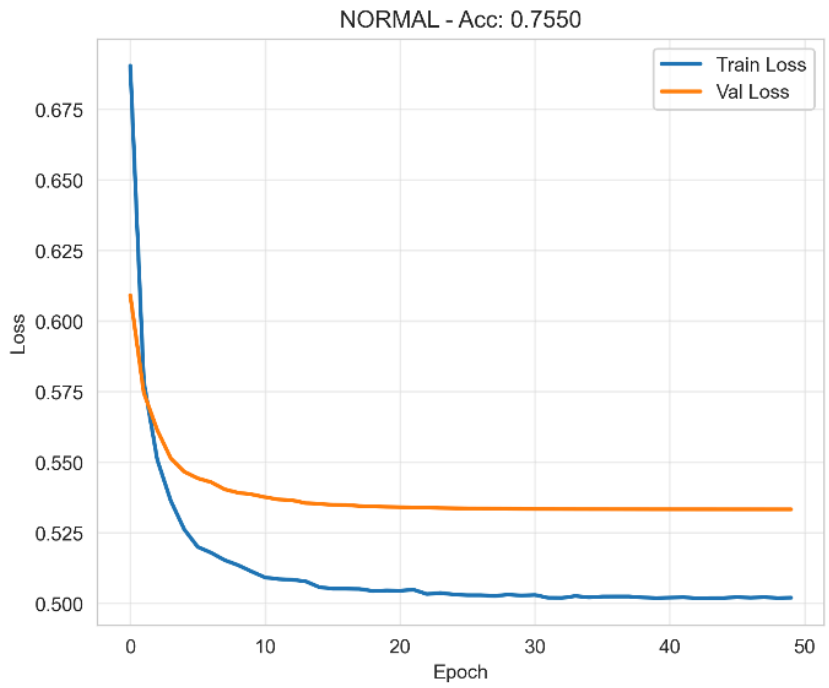
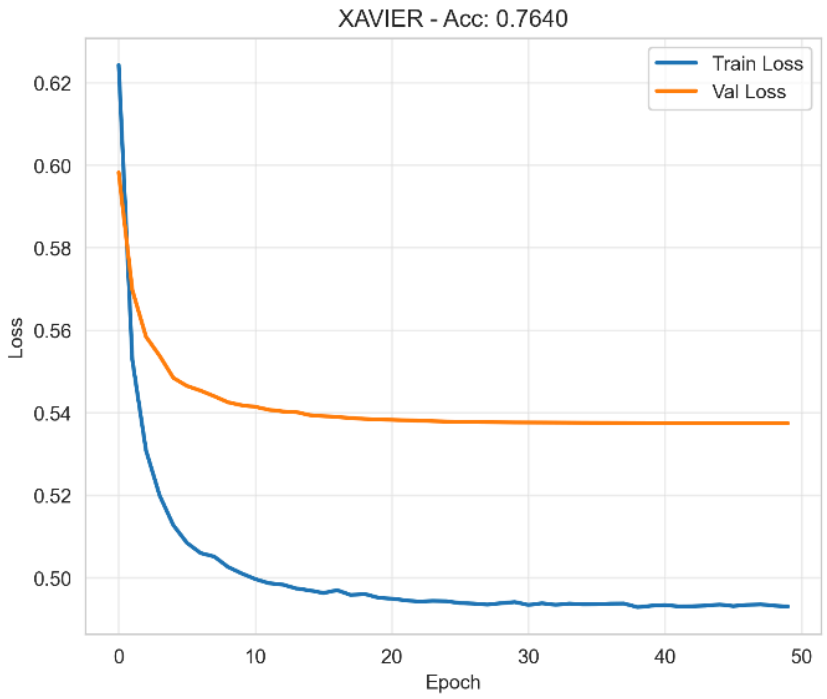
3.7 Pengaruh Inisialisasi Bobot

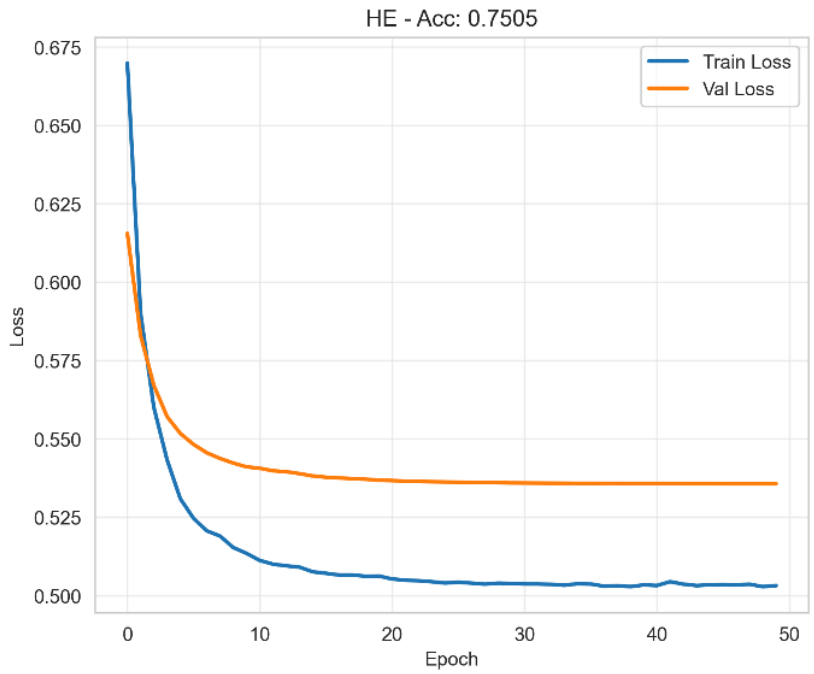
Arsitektur: [n_features, 32, 16, n_classes], aktivasi ReLU, learning rate 0.01, batch size 32, epochs 50.

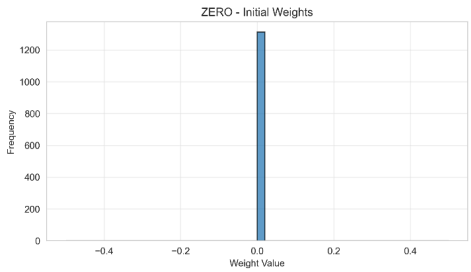
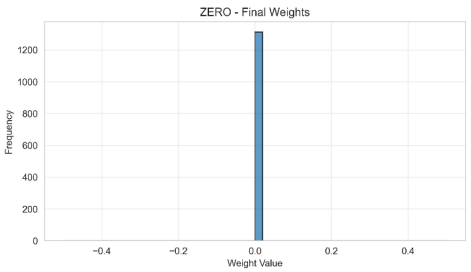
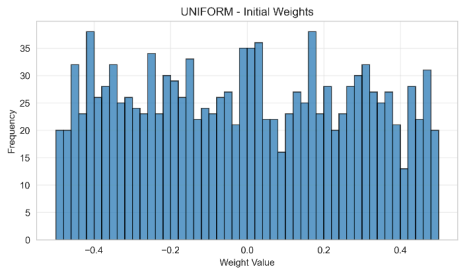
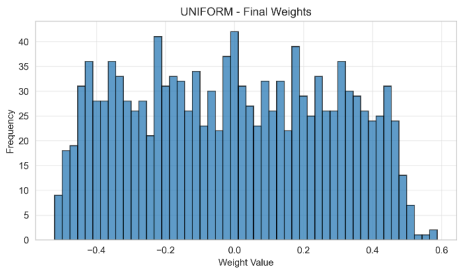
Initializer	Train Loss	Val Loss	Accuracy	Analisis
Zero	0.6661	0.6662	61.50%	Symmetry problem: semua neuron belajar hal sama karena gradien identik. Early stopping di epoch 19. Loss stagnan mendekati ln(2) ~ 0.693.
Uniform	0.5024	0.5475	75.10%	Inisialisasi acak memecah simetri. Rentang [-0.5, 0.5] menghasilkan konvergensi yang baik dan stabil.
Normal	0.5019	0.5333	75.50%	Distribusi Gaussian (mean=0, var=0.1) menghasilkan val loss lebih baik dari Uniform. Cocok untuk jaringan dalam.
Xavier	0.4930	0.5374	76.40%	Accuracy terbaik; skala bobot disesuaikan dengan fan_in dan fan_out, menjaga variansi signal stabil antar layer.
He	0.5032	0.5357	75.05%	Didesain untuk ReLU (2/fan_in); performa

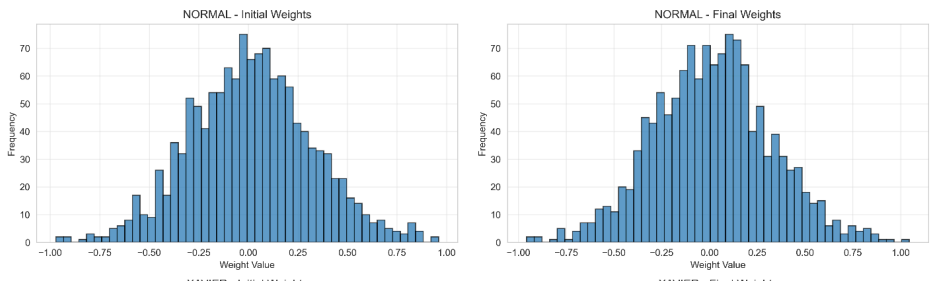
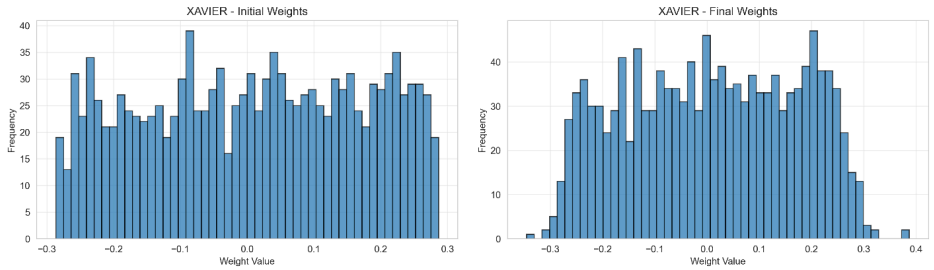
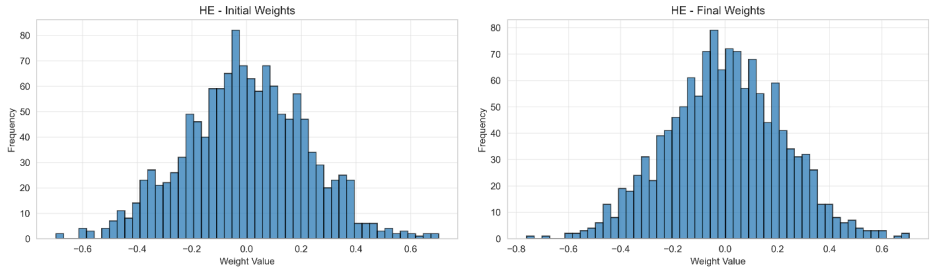
				baik namun tidak sebaik Xavier untuk dataset ini.

Initializer	Grafik training loss dan validation loss
Zero	
Uniform	

Normal	
Xavier	

He	

Initializer	Grafik Initial Weight dan Final Weight
Zero	 
Uniform	 

Normal	
Xavier	
He	

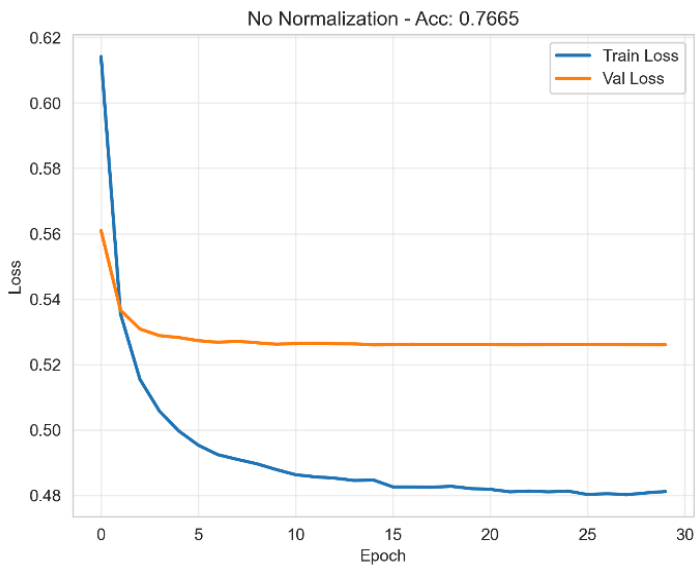
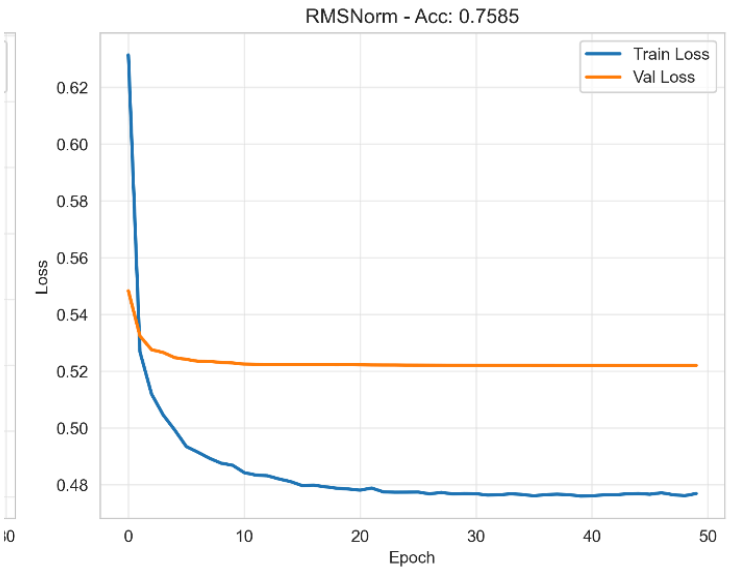
Analisis: Zero initialization terbukti tidak efektif karena symmetry problem, semua neuron dalam satu layer menghasilkan gradien identik sehingga bobot tidak pernah berbeda, menyebabkan model hanya memprediksi satu kelas (accuracy 61.50% setara tebak mayoritas). Xavier initialization memberikan accuracy terbaik (76.40%) karena menyesuaikan skala bobot berdasarkan jumlah unit input dan output, sehingga variansi sinyal terjaga stabil dari input hingga output. He initialization dirancang khusus untuk ReLU namun tidak lebih baik dari Xavier pada kasus ini, kemungkinan karena dataset yang relatif kecil dan sederhana tidak membutuhkan inisialisasi yang sangat agresif.

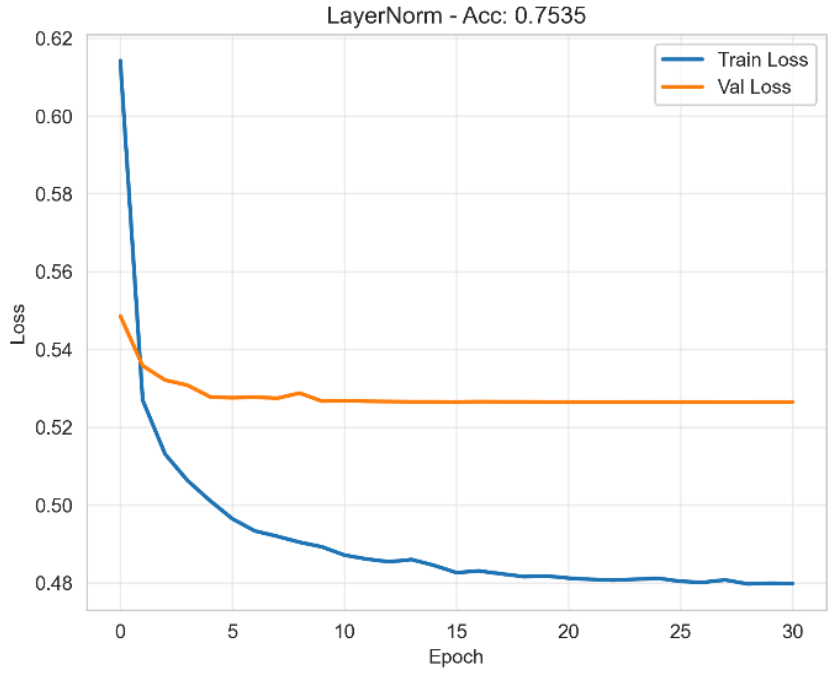
3.8 Pengaruh Normalisasi RMSNorm

Arsitektur: [n_features, 32, 16, n_classes], aktivasi ReLU, learning rate 0.01, batch size 32, epochs 50. Normalisasi diterapkan pada semua hidden layer (kecuali output layer).

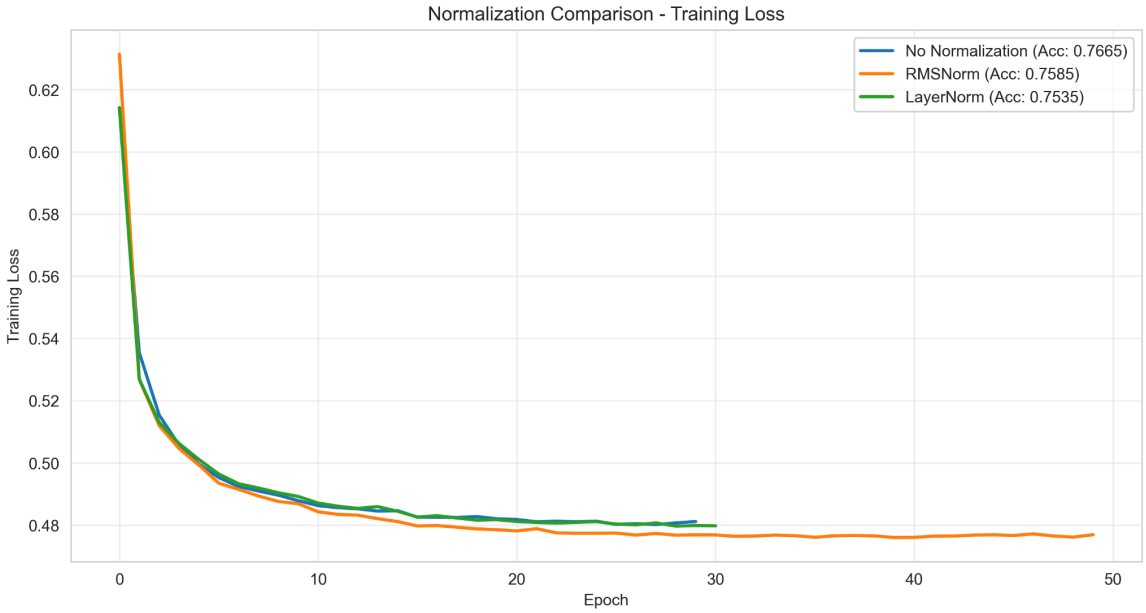
Kondisi	Train Loss	Val Loss	Accuracy	Keterangan
Tanpa Normalisasi	0.4811	0.5260	76.65%	Baseline; performa terbaik untuk dataset ini

Kondisi	Train Loss	Val Loss	Accuracy	Keterangan
RMSNorm [rmsnorm, rmsnorm, None]	0.4769	0.5220	75.85%	Train loss dan val loss lebih rendah, namun accuracy sedikit turun
LayerNorm [layernorm, layernorm, None]	0.4798	0.5264	75.35%	Val loss hampir sama dengan baseline; overhead normalisasi tidak memberikan manfaat signifikan

Kondisi	Grafik training loss dan validation loss
Tanpa Normalisasi	
RMSNorm [rmsnorm, rmsnorm, None]	

LayerNorm [layernorm, layernorm, None]	

Grafik perbandingan antar kondisi:



Analisis: RMSNorm memberikan train loss dan val loss yang sedikit lebih rendah dibanding tanpa normalisasi (0.4769 vs 0.4811 train loss; 0.5220 vs 0.5260 val loss), menunjukkan bahwa normalisasi membantu stabilisasi distribusi aktivasi antar layer. Namun secara accuracy, model tanpa normalisasi justru lebih tinggi (76.65% vs 75.85%). Hal ini kemungkinan terjadi karena dataset ini relatif kecil dan sudah di-scale dengan StandardScaler, sehingga normalisasi tambahan memberikan manfaat marginal bahkan sedikit mengganggu. LayerNorm memberikan hasil serupa dengan RMSNorm namun dengan overhead parameter lebih besar (memiliki parameter beta/shift

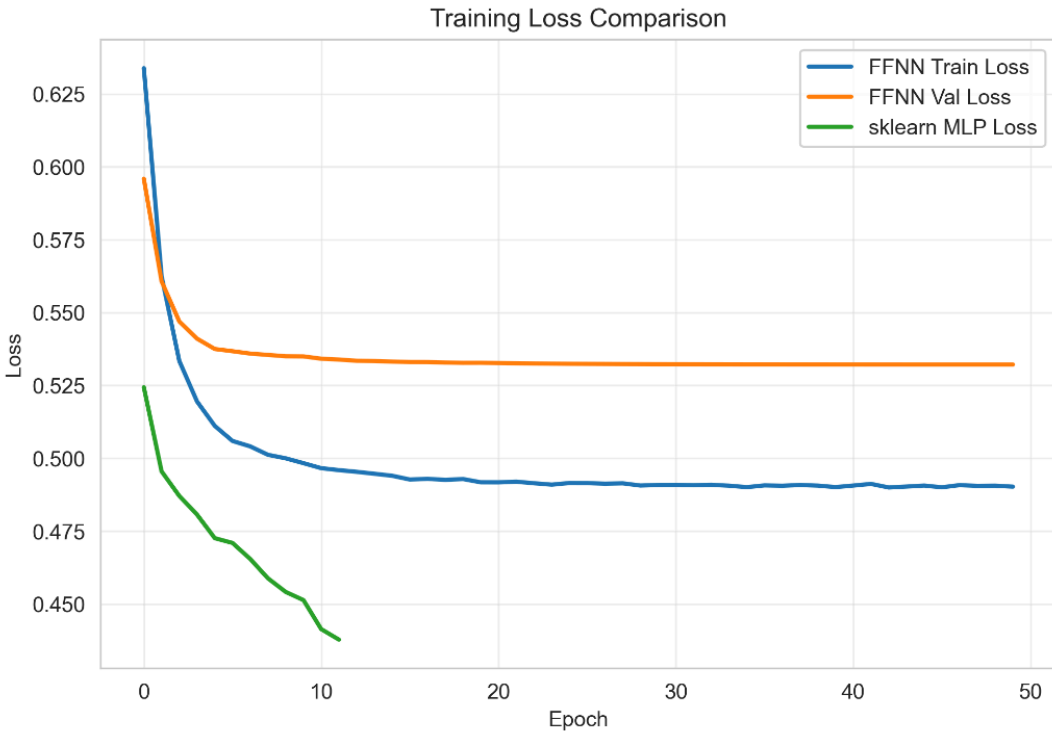
tambahan). RMSNorm lebih efisien secara komputasi karena hanya menghitung RMS tanpa menghitung mean.

3.9 Perbandingan dengan Sklearn MLP

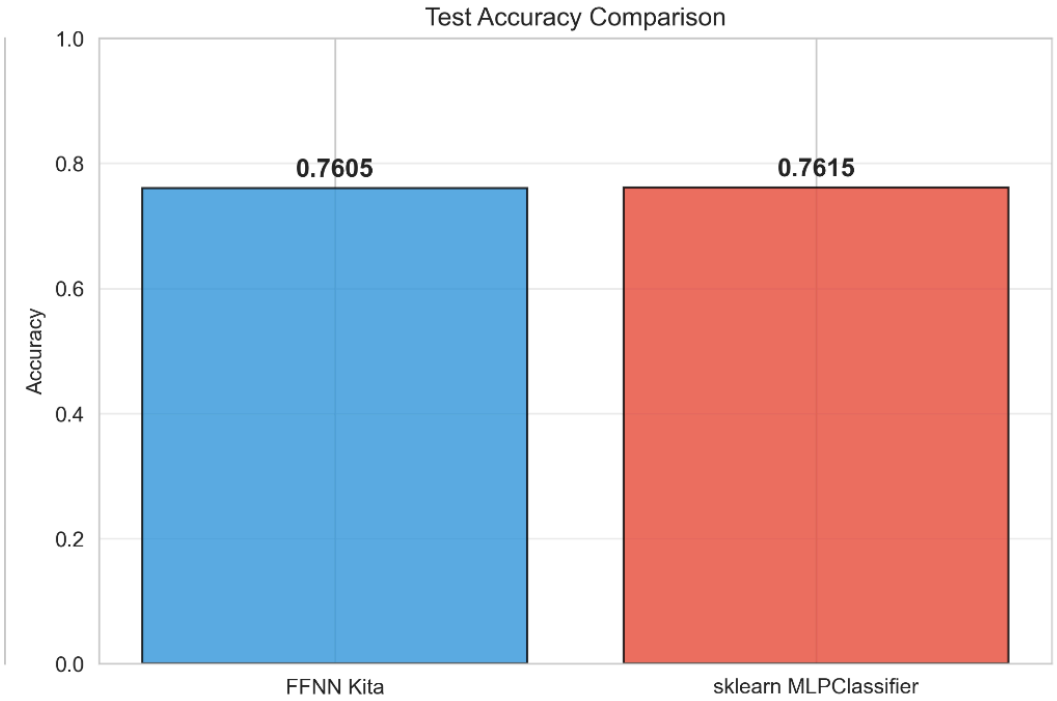
Perbandingan dilakukan dengan konfigurasi yang semirip mungkin. Arsitektur: [n_features, 32, 16, n_classes]. Aktivasi: ReLU. Epochs: 50. Learning rate: 0.01. Optimizer sklearn menggunakan Adam bawaan dengan early stopping (validation_fraction=0.2, n_iter_no_change=15).

Model	Accuracy	F1-Score	Keterangan
Implementasi Sendiri (FFNN)	76.30%	0.8154	Precision: 78.25%, Recall: 85.12%. Dilatih dengan GD + LR scheduler.
Sklearn MLPClassifier	76.35%	0.8197	Precision: 77.17%, Recall: 87.40%. Menggunakan Adam optimizer internal.

Grafik perbandingan training loss:



Grafik perbandingan test accuracy:



Analisis: Hasil menunjukkan performa yang sangat setara antara implementasi sendiri (76.30%) dan sklearn MLP (76.35%), dengan selisih hanya 0.05%. Ini membuktikan bahwa implementasi FFNN from scratch telah berhasil mereplikasi mekanisme neural network secara benar.

Perbedaan kecil yang ada dapat dijelaskan oleh beberapa faktor: (1) Sklearn menggunakan Adam optimizer secara default yang memiliki adaptive learning rate per parameter, sementara implementasi kita menggunakan Gradient Descent dengan LR scheduler sederhana; (2) Sklearn melakukan validasi internal (20% dari training set) untuk early stopping sehingga data yang digunakan sedikit berbeda; (3) Implementasi sklearn sudah sangat dioptimasi secara numerik (e.g., penggunaan BLAS untuk operasi matriks). Implementasi kita memiliki Precision lebih tinggi (78.25% vs 77.17%) meskipun Recall lebih rendah, menunjukkan model kita lebih konservatif dalam memprediksi kelas positif (Placed).

4. Kesimpulan dan Saran

4.1 Kesimpulan

- Keberhasilan implementasi FFNN from scratch beserta semua komponen wajib (forward propagation, backpropagation, berbagai fungsi aktivasi, loss function, inisialisasi bobot, regularisasi L1/L2, dan optimizer gradient descent) menggunakan hanya NumPy.
- Temuan utama dari eksperimen depth & width: penambahan depth (4 hidden layers) memberikan val loss terbaik (0.5295) namun perbedaan dengan model shallow tidak signifikan (~0.3%). Width 8 neuron justru memberikan accuracy terbaik di antara variasi width, mengindikasikan bahwa kapasitas berlebih tanpa regularisasi tidak selalu menguntungkan.
- Temuan dari eksperimen fungsi aktivasi: fungsi aktivasi terbaik untuk kasus ini adalah Linear (76.65%) karena data placement sudah memiliki representasi linear yang kuat setelah feature engineering. ReLU menjadi pilihan praktis terbaik dengan stabilitas konvergensi yang baik, sementara Sigmoid memiliki performa terburuk akibat vanishing gradient.
- Temuan dari eksperimen learning rate: nilai 0.01 optimal sebagai keseimbangan antara kecepatan konvergensi dan stabilitas. LR 0.001 terlalu lambat (loss masih 0.60 di epoch 50), sementara LR 0.1 lebih cepat namun rentan osilasi.
- Temuan dari eksperimen regularisasi: L2 dengan lambda=0.001 lebih efektif (76.55%) untuk kasus ini karena menjaga semua bobot kecil secara merata. Zero initialization terbukti tidak dapat digunakan karena symmetry problem.
- Perbandingan dengan sklearn MLP: implementasi sendiri mencapai performa setara (76.30% vs 76.35%) dengan selisih hanya 0.05%, membuktikan kebenaran implementasi.
- Adam optimizer memberikan train loss lebih rendah (0.4261) dibanding SGD (0.4700), mengkonfirmasi keunggulan adaptive learning rate. RMSNorm memberikan val loss lebih baik (0.5220) namun manfaatnya marginal pada dataset yang sudah di *scale*. Implementasi autodiff (Value class) berhasil melakukan diferensiasi otomatis menggunakan computational graph dan topological sort.

4.2 Saran

- Eksplorasi arsitektur yang lebih kompleks dengan kombinasi batch normalization dan residual connections untuk mengatasi vanishing gradient pada jaringan yang lebih dalam.
- Penggunaan Adam optimizer sebagai default dengan warmup learning rate schedule untuk konvergensi yang lebih stabil dan efisien.
- Implementasi early stopping yang lebih canggih dengan restore best weights telah diimplementasikan; dapat dikembangkan dengan learning rate warmup dan cosine annealing.
- Eksplorasi teknik feature selection yang lebih sistematis untuk mengurangi dimensi fitur tanpa kehilangan informasi penting.
- Pengujian pada dataset yang lebih besar dan beragam untuk menguji skalabilitas dan generalisasi implementasi.
- Optimasi performa komputasi menggunakan vectorized operations yang lebih efisien atau integrasi dengan library seperti CuPy untuk GPU acceleration.

Nama / NIM	Kontribusi (%)	Tugas yang Dikerjakan
Muhammad Raihaan Perdana (13523124)	35%	● Implementasi dan eksekusi semua skenario eksperimen di testing notebook (Eksperimen 1-10) ● Pengumpulan dan dokumentasi hasil eksperimen (training history, prediction CSV) ● Implementasi dan analisis perbandingan sklearn MLPClassifier ● Penulisan laporan Bab 3 (Eksperimen dan Hasil)
Danendra Shafi Athallah (13523136)	35%	● Setup struktur proyek dan base classes (BaseModel, BaseLayer, base aktivasi/loss/initializer/optimizer) ● Implementasi fungsi aktivasi dasar (Linear, ReLU, Sigmoid, Tanh, Softmax) dan bonus (LeakyReLU, ELU) ● Implementasi Dense layer dan backward propagation ● Implementasi loss functions (MSE, Binary Cross-Entropy, Categorical Cross-Entropy) ● Data preprocessing, feature engineering, dan EDA visualization ● Integrasi normalization layers (RMSNorm, LayerNorm) ke FFNN ● Pipeline training, hyperparameter tuning, dropout, learning rate scheduler ● Fix autodiff training dan Adam optimizer ● Implementasi perbandingan sklearn di main terminal
M. Abizzar Gamadrian (13523155)	30%	● Implementasi kelas FFNN utama (multi-layer, forward propagation, predict) ● Implementasi Value class untuk automatic differentiation (autodiff engine) ● Implementasi RMSNorm normalization ● Implementasi weight initializers bonus (Xavier dan He initialization) ● Implementasi regularizers L1 dan L2 ● Implementasi optimizer Gradient Descent dan perancangan Adam ● Implementasi Input layer, IO utilities (save/load), plotting, dan metrics ● Penulisan laporan Bab 1 dan Bab 2 (pendahuluan dan implementasi)

4.3 Daftar Pustaka

1. Karpathy, A. The spelled-out intro to neural networks and backpropagation: building micrograd. YouTube, 2022.
2. Osajima, J. Forward Propagation. https://www.jasonosajima.com/forwardprop
3. Osajima, J. Backpropagation. https://www.jasonosajima.com/backprop
4. NumPy Documentation v2.2. https://numpy.org/doc/2.2/
5. Scikit-learn: MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
6. Greenplace, E. The Softmax function and its derivative, 2016. https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/
7. Orr, D. Automatic Differentiation. https://douglasorr.github.io/2021-11-autodiff/article.html
8. Grosse, R. Tutorial 1: Automatic Differentiation. University of Toronto, 2022.
9. GeeksforGeeks. Training and Validation Loss in Deep Learning. https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/
10. BuiltIn. L2 Regularization. https://builtin.com/data-science/l2-regularization

4.4 Lampiran

Link Repository: https://github.com/danenftyessir/Min-Satu-Ardell_Tubes-1_IF3270.git
Sesuai dengan panduan penggunaan AI Institut Teknologi Bandung (Hal. 19), berikut adalah rincian penggunaan AI dalam pengerjaan tugas ini:

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, Kami:

Nama/NIM : Muhammad Raihaan Perdana/13523124,
Danendra Shafi Athallah/13523136,
M.Abizzar Gamadrian/13523155
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3270
Judul Tugas : Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa kami tidak menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan ?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	
Pembuatan Gambar, Video, dan/atau Grafik: Menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami Juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 16 Maret 2026

(Muhammad Raihaan Perdana)	(Danendra Shafi Athallah)	(M. Abizzar Gamadrian)
13523124	13523136	13523155